MosicaFE

A Unified Python Library for FEM, VEM and RT0 Discretizations of Partial Differential Equations

User Guide and Developer Manual

MosicaFE contributors

October 25, 2026

Abstract. This manual documents MosicaFE, a Python library that brings the finite element method (FEM), the virtual element method (VEM) and the lowest-order Raviart–Thomas mixed element (RT0×P0) under a single set of interfaces. The user writes only a few lines in the driver file: choose a mesh, choose an "element", choose a linear solver, choose a visualization tool, and a partial differential equation — Poisson, reaction-diffusion, Helmholtz or the RT0×P0 mixed formulation — is discretized and solved. The layered design (`MosicaFE.core`, `MosicaFE.spaces`, `MosicaFE.physics`, `MosicaFE.solvers`, `MosicaFE.postprocess`) makes both **adding a new element type** and **adding a new equation** a local modification. The manual contains complete usage examples, the mathematical background, verification of accuracy and convergence, and templates for extending the framework.

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Framework Philosophy . . . 1
- 1.3 Quick Start . . . 2
- 1.4 Package Structure . . . 3
- 1.5 Organization of This Manual . . . 4

2 Installation and Setup **5**
- 2.1 System Requirements . . . 5
  - 2.1.1 Python Version . . . 5
  - 2.1.2 Dependencies . . . 5
- 2.2 Installation . . . 5
  - 2.2.1 Method 1: Installing as a Package (Recommended) . . . 5
  - 2.2.2 Method 2: Using the Sources Directly . . . 5
- 2.3 Verifying the Installation . . . 6
- 2.4 Compiling This Manual . . . 6

3 Mathematical Background **7**
- 3.1 Model Problem . . . 7
- 3.2 Weak Formulation . . . 8
- 3.3 Finite Element Discretization (FEM) . . . 8
- 3.4 Arbitrary-Order VEM . . . 8
  - 3.4.1 Local Space and Degrees of Freedom . . . 9
  - 3.4.2 Scaled Monomials and Energy Projection . . . 9
  - 3.4.3 L^2 Projection, Stiffness, Mass and Load . . . 10
- 3.5 RT0×P0 Mixed Discretization . . . 11
- 3.6 Error Estimates . . . 11

4 Core Module: Mesh and Quadrature **12**
- 4.1 Mesh Data Structure . . . 12
  - 4.1.1 Mesh Components . . . 12
  - 4.1.2 VEM Mesh Generality and Refinement . . . 12
  - 4.1.3 Facets . . . 13
  - 4.1.4 Facet Topology and Orientation: `core/facets.py` . . . 13
- 4.2 Mesh Generation . . . 14
- 4.3 Mesh Properties and Geometric Quantities . . . 15
- 4.4 Mesh Refinement . . . 16
- 4.5 Reference Elements . . . 16
- 4.6 Quadrature Rules . . . 17

5 Discrete Spaces **19**
5.1 Space Abstraction . . . 19
5.2 Lagrange Elements (Classical Finite Elements) . . . 19
5.2.1 P1 Triangles . . . 19
5.2.2 P2 Triangles . . . 20
5.2.3 Tensor-Product Elements . . . 20
5.2.4 Global Numbering of the Degrees of Freedom . . . 20
5.3 Virtual Element Spaces . . . 20
5.4 RT0×P0 Mixed Space . . . 21
5.4.1 The Two Blocks Exposed by the Space Layer . . . 21
5.5 The Space Factory `FESpace` . . . 22

6 Physics Problems and Assembly **24**
6.1 The Physics Problem Interface . . . 24
6.2 The Poisson Problem . . . 24
6.3 The Helmholtz Problem . . . 25
6.4 Boundary Conditions . . . 26
6.4.1 Imposing Dirichlet Conditions Correctly . . . 26
6.4.2 Other Boundary Conditions . . . 27
6.5 Assembly Algorithm . . . 27
6.6 Using the Poisson Problem . . . 28
6.7 The PDE Registry . . . 28

7 Linear Solvers **29**
7.1 The Solver Interface . . . 29
7.2 Direct Methods . . . 29
7.3 Iterative Methods . . . 29
7.4 Solver Selection Guide . . . 30

8 Error Analysis and Convergence **31**
8.1 Error Norms . . . 31
8.2 Convergence Study Framework . . . 31
8.3 Convergence Rate Estimation . . . 32
8.4 Measured Verification Results . . . 32

9 Visualization **34**
9.1 Overview of the Plotting Tools . . . 34
9.2 Plotting Solutions . . . 34
9.3 Plotting Convergence Curves . . . 35

10 Complete Examples **36**
10.1 Example 1: Solving Poisson in Two Lines . . . 36
10.2 Example 2: Changing the Element (One Code, Three Methods) . . . 36
10.3 Example 3: VEM in 2D and 3D . . . 37
10.3.1 Fine-Mesh Results and Error Plots . . . 38
10.4 Example 4: The Low-Frequency Helmholtz Equation . . . 42
10.5 Example 5: The Reaction-Diffusion Equation . . . 43

11 Advanced Topics **44**
11.1 Accuracy Pitfalls and Lessons Learned . . . 44
11.1.1 Pitfall 1: Dirichlet Elimination Without the Right-Hand-Side Correction 44
11.1.2 Pitfall 2: Insufficient Quadrature Order for Tensor-Product Elements . . 44
11.1.3 Pitfall 3: Fixing the Constant in High-Order Projections . . . 44
11.1.4 Pitfall 4: Treating Moments as Point Values . . . 44
11.1.5 Pitfall 5: Repeated Indices in `numpy` Assignment . . . 45
11.2 Mesh Refinement Strategies . . . 45
11.3 Performance and Vectorization . . . 45
11.4 Relation to Other Libraries . . . 45

12 Extending the Framework **46**
12.1 Adding a New Element Type . . . 46
12.2 Adding a New Equation . . . 47
12.3 Adding a New Mixed Formulation (RT0×P0 Template) . . . 48
12.4 Adding a New Mesh Type . . . 48
12.5 Development Workflow and Testing Requirements . . . 49

13 Troubleshooting and FAQ **50**
13.1 Common Issues . . . 50
13.1.1 Import Errors . . . 50
13.1.2 Wrong Convergence Order . . . 50
13.1.3 Iterative Solver Does Not Converge . . . 50
13.1.4 Out of Memory . . . 51
13.2 Frequently Asked Questions . . . 51

A API Reference **52**
A.1 Top-Level Module `MosicaFE.` . . . 52
A.2 `Mesh` . . . 52
A.3 `FESpace` and the Space Methods . . . 53
A.4 Physics Problems . . . 54
A.5 Solvers and Postprocessing . . . 54

B Mathematical Notation **56**
B.1 General Notation . . . 56
B.2 Function Spaces . . . 56
B.3 Operators and Norms . . . 56

C Bibliography **58**

Chapter 1

Introduction

1.1 Overview

MosicaFE is a partial differential equation solver aimed at teaching and research. It merges three previously independent programs (classical finite elements, virtual elements, and RT0 mixed finite elements) into **a single package with a single set of interfaces**, sharing one mesh representation, one set of solvers and one set of postprocessing tools.

The core design goals of the library are:

(1) **Minimal call syntax**: driver files are usually shorter than 10 lines;

(2) **Decoupled modules**: mesh, element, equation, linear algebra and postprocessing are mutually independent;

(3) **Easy extension**: a new element or a new PDE requires implementing only a small number of interfaces;

(4) **Verifiable**: convergence studies and error norms are built in, and the observed convergence orders can be reproduced with one command;

(5) **Self-contained**: the only hard dependency is `numpy` (`scipy` and `matplotlib` are optional).

1.2 Framework Philosophy

The design of MosicaFE mirrors the mathematical structure of the finite element method:

$$\underbrace{\text{PDE}}_{\text{physics}} \xrightarrow{\text{weak form}} \underbrace{\text{variational problem}}_{\text{spaces}} \xrightarrow{\text{discretization}} \underbrace{Au = b}_{\text{assembly}} \xrightarrow{\text{solve}} \underbrace{u_h}_{\text{postprocess}} \tag{1.1}$$

Each step is the responsibility of a separate module, and modules communicate only through explicit interfaces:

- `MosicaFE.core` provides meshes and quadrature rules;
- `MosicaFE.spaces` translates "which element" into a **matrix-level interface**: stiffness matrix, mass matrix, load vector (see Chapter 5);
- `MosicaFE.physics` only takes linear combinations of those matrices, so a new PDE usually takes a dozen lines;

- `MosicaFE.solvers` is responsible for linear algebra;
- `MosicaFE.postprocess` is responsible for errors, convergence and visualization.

Key Point

The orthogonal decomposition of "element" and "equation" is what distinguishes MosicaFE most clearly from the three original programs. Those programs hard-wired "triangle mesh + linear element + Poisson + `spsolve` + plotting" into a single file; here these concerns are independent: changing the element does not require touching the equation, and changing the equation does not require touching the element.

1.3 Quick Start

Here is a complete 2D Poisson solver:

```
import numpy as np
from MosicaFE import Mesh, solve_poisson

source = lambda x: 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

u = solve_poisson(Mesh.rectangle(nx=32, ny=32), element="P2", f=source)
print("degrees of freedom:", u.size)
```

Listing 1.1: Solving the Poisson equation in two lines

The equivalent "step-by-step" form (which is also the standard form used throughout this manual):

```
import numpy as np
from MosicaFE import Mesh, FESpace, PoissonProblem
from MosicaFE.postprocess import plot_solution

def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def source(x):
    return 2*np.pi**2 * exact(x)

mesh = Mesh.rectangle(nx=32, ny=32)          # (1) choose a mesh
V = FESpace.lagrange(mesh, degree=2)         # (2) choose an element
problem = PoissonProblem(V, f=source, g=lambda x: 0.0)
u = problem.solve(method="direct")           # (3) choose a solver
plot_solution(u, mesh, V, filename="p2.png")  # (4) choose a visualization

print(problem.compute_errors(u, exact))
```

Listing 1.2: Choosing mesh / element / solver / visualization

Replacing `FESpace.lagrange(mesh,degree=2)` in Listing 1.2 by `FESpace.vem(mesh)` no longer produces a finite element solution: it produces the virtual element solution of the **same equation** on the **same mesh**, and nothing else in the program changes.

Changing the equation costs one line as well. Built-in equations are the Poisson equation, the reaction-diffusion equation, the Helmholtz equation and the RT0×P0 mixed formulation; each of them has a named constructor and a one-line helper:

```python
from MosicaFE import Mesh, PoissonProblem, HelmholtzProblem, solve_helmholtz

mesh = Mesh.rectangle(nx=32, ny=32)

# -Laplacian(u) = f           (k = 0 limit of the line below)
u = PoissonProblem(V, f=source, g=zero).solve()

# -Laplacian(u) - k^2 u = f   (constant wavenumber k, Chapter 6.3)
u = HelmholtzProblem(V, f=source, g=zero, k=1.0).solve()
u = solve_helmholtz(mesh, element="P2", f=source, k=1.0)     # one line
```

Listing 1.3: Changing the equation

Key Point

This is the reason the library exists: changing the discretization method costs one line of code.

1.4 Package Structure

Table 1.1: Module layout of MosicaFE

Module	Responsibility
`MosicaFE.core`	Mesh data structures and generation (interval / rectangle / box / polygon / Voronoi), **facet topology and orientation** (`facets.py`, the basis of $H(\mathrm{div})$-conforming spaces), quadrature rules, geometric tools (area, volume, Jacobian, gradient transformation)
`MosicaFE.spaces`	Discrete spaces: `LagrangeSpace` (P1/P2/Q1/Q2), `VEMSpace` (arbitrary-order enhanced VEM), `RT0Space` (RT0×P0), and the factory `FESpace`
`MosicaFE.physics`	PDE weak forms and assembly: `PoissonProblem`, `MixedPoissonProblem`, `ReactionDiffusionProblem`, `HelmholtzProblem`, plus a PDE registry
`MosicaFE.solvers`	Linear solvers: direct methods, CG, GMRES, BiCGSTAB, MINRES and automatic selection
`MosicaFE.postprocess`	Error norms (L^2 / H^1 / L^∞), convergence studies, visualization, and error norms for mixed formulations (pressure + flux)

Key Point

All three element families (FEM, VEM and RT0) are **implemented inside the library itself**: the explicit RT0×P0 basis functions, the exact mass matrix, the facet topology and the saddle-point assembly live in `spaces/rt0.py`, `core/facets.py` and `physics/mixed_poisson.py`, respectively. MosicaFE neither depends on nor requires any external finite element backend.

1.5 Organization of This Manual

- Chapter 2: installation and environment;
- Chapter 3: mathematical background of the three methods;
- Chapters 4–5: meshes, quadrature and discrete spaces;
- Chapters 6–7: equation assembly and linear solution;
- Chapters 8–9: error analysis and visualization;
- Chapter 10: complete examples;
- Chapters 11–12: advanced topics and framework extension;
- Chapter 13: frequently asked questions;
- Appendices A–C: API reference, mathematical notation, bibliography.

Chapter 2

Installation and Setup

2.1 System Requirements

2.1.1 Python Version

Python $\geq$ 3.9 is required. The environment used for development and verification is Python 3.12.10 (conda environment `fealpy`), with `numpy` 2.2.4, `scipy` 1.15.2 and `matplotlib` 3.10.1.

2.1.2 Dependencies

Table 2.1: Dependencies

Package	Required	Purpose
`numpy`	yes	arrays, dense linear algebra
`scipy`	optional	sparse matrices, `spsolve`, Krylov solvers, Voronoi meshes
`matplotlib`	optional	visualization

If `scipy` is missing, MosicaFE automatically falls back to a purely `numpy`-based dense implementation (handled uniformly in `MosicaFE._compat`), and small problems remain solvable. If `matplotlib` is missing, all plotting functions print a notice and return `None` instead of raising an exception.

2.2 Installation

2.2.1 Method 1: Installing as a Package (Recommended)

```
cd D:\mosica\MosicaFE
python -m pip install -e .
```

2.2.2 Method 2: Using the Sources Directly

It is enough to put the **parent directory** of MosicaFE on the module search path; no installation is needed:

```
# PowerShell
cd D:\mosica
$env:PYTHONPATH = "D:\mosica"
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\examples\poisson_demo.py
```

2.3 Verifying the Installation

MosicaFE ships two scripts that do not depend on a third-party test framework:

```
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_tests.py
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_checks.py
```

The first runs 106 unit and integration tests; the second prints the measured convergence orders of the various methods. An excerpt of the expected output:

```
=== P2 ===
  level        h                 L2    rate          H1    rate        Linf    rate
      0  0.35355    3.4643e-02       -  4.4391e-01       -  1.4645e-01       -
      1  0.17678    4.4076e-03    2.97  1.1632e-01    1.93  3.8060e-02    1.94
      2  0.08839    5.5040e-04    3.00  2.9416e-02    1.98  9.6074e-03    1.99
      3  0.04419    6.8699e-05    3.00  7.3752e-03    2.00  2.4076e-03    2.00
  rate[L2] (last 3) = 3.002
  rate[H1] (last 3) = 1.990
```

If `pytest` is installed, one may also run

```
python -m pytest MosicaFE/tests -q
```

2.4 Compiling This Manual

The document uses the standard `book` class and contains no CJK text, so pdfLaTeX is sufficient; compile twice to obtain the table of contents and the cross-references:

```
pdflatex MosicaFE_Guidebook.tex
pdflatex MosicaFE_Guidebook.tex
```

Alternatively use `latexmk -pdf MosicaFE_Guidebook.tex`. XeLaTeX produces the same output. TeXShop users select `pdfLaTeX` as the engine and press Typeset.

Chapter 3

Mathematical Background

3.1 Model Problem

The model problem solved by the library (and by the three original programs) is the Poisson equation

$$\begin{cases} -\nabla \cdot (\kappa \nabla u) = f, & \text{in } \Omega, \\ u = g, & \text{on } \partial\Omega, \end{cases} \tag{3.1}$$

where $\Omega \subset \mathbb{R}^d$ ($d = 1, 2, 3$) is a bounded domain and $\kappa > 0$ is the diffusion coefficient. For $g = 0$ and $\kappa = 1$ this is the classical problem $-\Delta u = f$.

The Helmholtz model problem. The second model problem is the (frequency-domain, time-harmonic) **Helmholtz equation** with a *constant* wavenumber k and a constant coefficient κ,

$$\begin{cases} -\nabla \cdot (\kappa \nabla u) - k^2 u = f, & \text{in } \Omega, \\ u = g, & \text{on } \partial\Omega. \end{cases} \tag{3.2}$$

It is the equation solved by the built-in `HelmholtzProblem` and by the one-line helper `solve_helmholtz`. Two remarks are needed.

Key Point

Low frequency means non-resonant. The operator is $-\Delta - k^2$. On a bounded domain with Dirichlet data, $-\Delta$ has a discrete spectrum $0 < \lambda_1 \le \lambda_2 \le \dots$; the solution of (3.2) is unique and the bilinear form $a(u, v) = \kappa(\nabla u, \nabla v) - k^2(u, v)$ is positive definite — and so is the discrete matrix $A = \kappa K - k^2 M$ — if and only if $k^2 < \lambda_1$. On the unit square $\lambda_1 = 2\pi^2 \approx 19.74$, so the low-frequency range is $k < \sqrt{2}\,\pi \approx 4.443$. As soon as k^2 crosses λ_1 (or any other eigenvalue) the matrix becomes **symmetric indefinite** and ill-conditioned: the linear system must be solved by `"direct"` or `"minres"`, and the error is amplified. A discrete, mesh-dependent version of the criterion is $\lambda_{1,h}$, the smallest eigenvalue of the generalized problem $Kv = \lambda M v$ on the interior degrees of freedom; by the Galerkin theory $\lambda_{1,h} \ge \lambda_1$ and $\lambda_{1,h} \to \lambda_1$ as $h \to 0$.

Caution

A second condition has no analogue for the Poisson problem. The discrete wave must be resolved by the mesh: the phase error (numerical dispersion) of the scheme is controlled by kh, and for $kh \gtrsim 1$ the error is dominated by *pollution* rather than by interpolation. Refining the mesh only restores the optimal order once $kh \lesssim 1$; a convergence table for the Helmholtz problem must therefore always be read together with the kh column (Chapter 6 assembles the equation, Chapter 10 reports the measured orders).

The wavenumber is written k (lower case) throughout this manual to avoid a clash with the symbol K for a mesh cell (Appendix B).

3.2 Weak Formulation

Definition 3.1 (Weak formulation)**.** Let $V = \{v \in H^1(\Omega) : v|_{\partial\Omega} = 0\}$ and $V_g = \{v \in H^1(\Omega) : v|_{\partial\Omega} = g\}$. Find $u \in V_g$ such that

$$a(u, v) = \ell(v), \qquad \forall v \in V, \tag{3.3}$$

where

$$a(u, v) = \int_\Omega \kappa \nabla u \cdot \nabla v \, \mathrm{d}x, \qquad \ell(v) = \int_\Omega f v \, \mathrm{d}x. \tag{3.4}$$

The right-hand side ℓ may also carry a Neumann data term $\int_{\Gamma_N} h v \, \mathrm{d}s$; the physics layer of MosicaFE reserves an extension point for it.

3.3 Finite Element Discretization (FEM)

Let $V_h \subset V$ be a finite-dimensional subspace with basis $\{\varphi_i\}_{i=1}^n$. The discrete problem reads: find $u_h = \sum_j u_j \varphi_j$ such that

$$\sum_{j=1}^{n} u_j \, a(\varphi_j, \varphi_i) = \ell(\varphi_i), \qquad i = 1, \dots, n, \tag{3.5}$$

that is, the linear system

$$A u = b, \qquad A_{ij} = \int_\Omega \kappa \nabla \varphi_j \cdot \nabla \varphi_i \, \mathrm{d}x, \qquad b_i = \int_\Omega f \varphi_i \, \mathrm{d}x. \tag{3.6}$$

Computations on the reference element $\hat{K}$ make use of the gradient transformation

$$\nabla_x \varphi_i = J_K^{-\top} \nabla_{\hat{x}} \hat{\varphi}_i, \qquad \int_K g \, \mathrm{d}x = \int_{\hat{K}} g \circ F_K \, |\det J_K| \, \mathrm{d}\hat{x}. \tag{3.7}$$

3.4 Arbitrary-Order Virtual Element Discretization (VEM)

The library implements conforming enhanced VEM in two and three dimensions. A positive integer `degree=k` selects the order. The mesh–space–problem–solve–postprocess workflow is unchanged. `LocalVEM` in `spaces/vem_local.py` builds the local operators recursively; `VEMSpace` in `spaces/vem.py` shares global degrees of freedom and assembles the system. Explicit virtual basis functions are unnecessary.

Table 3.1: Lagrange elements available in MosicaFE

Element	Name	Local DOFs per cell	DOF type
triangle	P1	3	vertices
triangle	P2	6	vertices + edge midpoints
quadrilateral	Q1	4	vertices
quadrilateral	Q2	9	vertices + edge midpoints + cell centre
tetrahedron	P1	4	vertices
tetrahedron	P2	10	vertices + edge midpoints
hexahedron	Q1	8	vertices
hexahedron	Q2	27	vertices + edges + faces + cell interior

3.4.1 Local Space and Degrees of Freedom

Let $\mathcal{P}_r(E)$ contain polynomials of total degree at most r on an entity E, with $\mathcal{P}_r = \{0\}$ for $r < 0$. Let $\mathcal{B}_k^*(K)$ contain the scaled monomials of total degree exactly $k-1$ or k. Cells and, recursively, 3D faces use the enhanced space

$$\begin{aligned} V_k(K) = \{v \in H^1(K) : v|_{\partial K} \text{ continuous}, \quad v|_F \in V_k(F), \quad \Delta v \in \mathcal{P}_k(K), \\ (v - \Pi_k^\nabla v, m)_K = 0 \quad \forall m \in \mathcal{B}_k^*(K)\}. \end{aligned} \tag{3.8}$$

In 2D, F is an edge and $V_k(F) = \mathcal{P}_k(F)$; in 3D, F is a planar polygonal face. Enhancement supplies the high moments without extra unknowns. The degrees of freedom are:

1. Vertex values and values at $k-1$ equally spaced interior points of each edge.
2. Normalized moments $|E|^{-1}\int_E v m_\alpha$, $|\alpha| \le k-2$, on 2D cells and on 3D faces.
3. The analogous volume moments on 3D cells, $|\alpha| \le k-2$.

Shared vertices, edges and faces receive a single global set of unknowns. Only vertex values remain when $k = 1$. For n_v vertices, n_e edges and n_f faces,

$$\begin{aligned} N_K^{2D} &= n_v + n_e(k-1) + \frac{k(k-1)}{2}, \\ N_K^{3D} &= n_v + n_e(k-1) + n_f\frac{k(k-1)}{2} + \frac{(k-1)k(k+1)}{6}. \end{aligned}$$

3.4.2 Scaled Monomials and Energy Projection

`powers(dim,k)` enumerates $|\alpha| \le k$, so no separate formula is needed for each order. The center c_E is the arithmetic mean of the vertices, h_E is the diameter, and $m_\alpha(x) = \prod_j (((x - c_E)Q_E)_j/h_E)^{\alpha_j}$. For 3D faces, Q_E is an orthonormal plane basis obtained by SVD; volume cells use the coordinate axes. The energy projection satisfies

$$(\nabla\Pi_k^\nabla v, \nabla q)_K = -(v, \Delta q)_K + \sum_{F\subset\partial K}(v, \partial_n q)_F, \qquad q \in \mathcal{P}_k(K). \tag{3.9}$$

The internal moments determine the volume term since $\deg \Delta q \le k-2$. Child projections determine the boundary term. On an edge the $k+1$ point values uniquely determine the polynomial trace:

$$v|_e = \sum_{i=0}^{k} v(x_i)\ell_i, \qquad \int_e v\,\partial_n q = \sum_{i=0}^{k} v(x_i)\int_e \ell_i\,\partial_n q. \tag{3.10}$$

$$\begin{cases} \displaystyle\int_K \Pi_k^\nabla v = \int_K v, & k \geq 2, \\ \displaystyle\frac{1}{n_v}\sum_i (\Pi_1^\nabla v)(X_i) = \frac{1}{n_v}\sum_i v(X_i), & k = 1. \end{cases} \tag{3.11}$$

The last condition fixes the undetermined constant. Set $D_{i\alpha} = \mathrm{dof}_i(m_\alpha)$, $H_{\alpha\beta} = (m_\alpha, m_\beta)_K$, and $G_{\alpha\beta} = (\nabla m_\alpha, \nabla m_\beta)_K$. Replacing the constant row by the mean condition gives $\widetilde{G}$ and the corresponding right-hand side B:

$$P_\nabla = \mathrm{solve}(\widetilde{G}, B), \qquad \Pi_k^\nabla v = \sum_\alpha (P_\nabla \mathbf{v})_\alpha m_\alpha. \tag{3.12}$$

This is a linear solve, replacing the old pseudoinverse or linear least-squares reconstruction. The identity $P_\nabla D = I$ verifies reproduction of all of $\mathcal{P}_k$.

3.4.3 L^2 Projection, Stiffness, Mass and Load

Start with $C = HP_\nabla$, then replace rows of degree at most $k-2$ by $|K|$ times the selection rows for the true internal moments. Higher rows retain the enhanced moments. Thus $P_0 = \mathrm{solve}(H, C)$.

$$A_K = P_\nabla^T G P_\nabla + S_K, \tag{3.13}$$

$$R_\nabla = I - DP_\nabla, \qquad S_K = \sigma R_\nabla^T R_\nabla, \tag{3.14}$$

$$\sigma = \frac{\mathrm{tr}(P_\nabla^T G P_\nabla)}{\max(\mathrm{tr}(R_\nabla^T R_\nabla), 10^{-30})}. \tag{3.15}$$

$$\begin{aligned} M_K &= P_0^T H P_0 + |K|(I - DP_0)^T(I - DP_0), \\ b_K &= P_0^T \mathbf{f}_K, \qquad (\mathbf{f}_K)_\alpha = \int_K f m_\alpha. \end{aligned} \tag{3.16}$$

The default `trace` stabilization is symmetric positive semi-definite; `legacy` is now a compatibility alias. Stabilization is skipped when the residual norm is below 10^{-10}. `none` returns only the consistency term and generally leaves an uncontrolled projection kernel; use it for diagnostics. The central matrix operations are:

```
energy = np.linalg.solve(system, rhs)
moments = H @ energy
for j in range(len(self.low)):
    moments[j] = 0.0
    moments[j, self.moment_start+j] = self.measure
l2 = np.linalg.solve(H, moments)
consistent = energy.T @ G @ energy
residual = np.eye(len(self.dofs)) - D @ energy
stable = residual.T @ residual
scale = np.trace(consistent) / max(np.trace(stable), 1e-30)
```

Listing 3.1: Core local projection and stabilization operations

The default quadrature parameter is $2k+2$, with dynamic Gauss/Duffy rules at higher order. Triangle/tetrahedron fans serve integration only and introduce no FEM submesh unknowns. `VEMSpace` reuses `BaseFESpace` assembly. Dirichlet data sets both point values and normalized boundary-face moments; a moment degree of freedom must not be treated as a point value.

3.5 RT0×P0 Mixed Discretization

The mixed form writes the Poisson equation as a first-order system

$$q = -\nabla p, \qquad \nabla \cdot q = f \ \text{ in } \Omega, \qquad p = 0 \ \text{ on } \partial\Omega. \tag{3.17}$$

Weak formulation: find $(q, p) \in RT0 \times P0$ such that

$$\int_\Omega \nu \cdot q \,\mathrm{d}x - \int_\Omega p \,\nabla \cdot \nu \,\mathrm{d}x = 0, \qquad \int_\Omega r \,\nabla \cdot q \,\mathrm{d}x = \int_\Omega f r \,\mathrm{d}x \tag{3.18}$$

for all (ν, r). After discretization one obtains the saddle-point system

$$\begin{pmatrix} M & B^\top \\ B & 0 \end{pmatrix} \begin{pmatrix} \mathbf{Q} \\ \mathbf{P} \end{pmatrix} = \begin{pmatrix} 0 \\ \mathbf{f} \end{pmatrix}, \tag{3.19}$$

which is **symmetric indefinite**; the conjugate gradient method therefore does not apply.

Table 3.2: Degrees of freedom and boundary conditions of RT0×P0

Quantity	Location of DOFs	Count
flux q_h	one per cell facet (2D edge / 3D triangular face), shared by neighbouring cells	#facets
pressure p_h	one constant per cell	#cells
Dirichlet condition $p = 0$	**natural** boundary condition of the mixed form, no elimination needed	

3.6 Error Estimates

Theorem 3.1 (Céa's lemma)**.** *Let u be the exact solution and u_h the finite element solution. Then $\|u - u_h\|_V \le C \inf_{v_h \in V_h} \|u - v_h\|_V$.*

Theorem 3.2 (A priori error estimate)**.** *If $u \in H^{k+1}(\Omega)$ and polynomials of degree k are used, then*

$$\|u - u_h\|_{L^2} \le C h^{k+1} |u|_{H^{k+1}}, \qquad |u - u_h|_{H^1} \le C h^k |u|_{H^{k+1}}. \tag{3.20}$$

Example 3.1 (Measured convergence orders of this library)**.**

- P1 (triangles): L^2 is $O(h^2)$, H^1 is $O(h)$;
- P2 / Q2: L^2 is $O(h^3)$, H^1 is $O(h^2)$;
- VEM ($k \ge 1$, admissible polygons/polyhedra): under regularity and stability assumptions, L^2 is $O(h^{k+1})$ and H^1 is $O(h^k)$;
- RT0×P0: pressure L^2 is $O(h)$, flux L^2 is $O(h)$.

Chapter 8 reports the measured data.

Chapter 4

Core Module: Mesh and Quadrature

4.1 Mesh Data Structure

4.1.1 Mesh Components

A `Mesh` object stores the following information:

Table 4.1: Main attributes of `Mesh`

Attribute	Type	Meaning
`nodes`	`(n_nodes, dim)` array	vertex coordinates
`elements`	`list[ ndarray]`	vertex indices of each cell, **ragged**
`element_type`	`str`	`interval/triangle/quad/tet/hex/polygon/polyhedron`
`boundary_ nodes`	`ndarray`	boundary vertex indices (determined automatically from the facet topology)
`dim`	`int`	spatial dimension

Key Point

`elements` is deliberately a **ragged list**: triangles, quadrilaterals, pentagons, Voronoi polygons, tetrahedra and hexahedra can therefore share a single assembly loop. When all cells have the same number of vertices, a `(n_cells,k)` array view can be obtained through `mesh.uniform_vertices`.

4.1.2 VEM Mesh Generality and Refinement

Ragged `elements` supports cells with different vertex counts. In 3D, `faces` accepts either a common `faces[face][vertex]` template or per-cell `faces[cell][face][vertex]` tables, using cell-local vertex indices. Explicit faces take precedence over inference by vertex count: eight vertices may describe a heptagonal pyramid rather than a hexahedron. `mesh.get_cell_ faces(c)` returns the actual cell faces.

```
mesh2 = Mesh.voronoi_polygon(n_points=1024, seed=42)
mesh3 = Mesh.voronoi_polyhedron(n_points=192, seed=42)
# Common template: faces[face][vertex].
# Per-cell faces: faces[cell][face][vertex].
```

```
# mesh3 = Mesh(nodes, elements, element_type="polyhedron", faces=faces)
```

Listing 4.1: Polygonal and polyhedral mesh entry points

Current geometric requirements: 2D cells and 3D faces are straight-sided, planar polygons whose vertex mean lies strictly inside their star kernel. The same star-shaped condition applies to 3D volumes. A volume boundary must be closed, connected and orientable, with exactly two faces incident on each surface edge. Concave cells satisfying these requirements are allowed. A shared face must have identical global vertices and the same boundary cycle, with at most two owning cells. Edge adjacency propagates face orientation; signed volume selects outward normals. Holes, curved edges and nonplanar faces are unsupported; cracks, overlaps and hanging nodes are not repaired automatically.

The 2D Voronoi generator uses eight mirrored directions; the 3D generator uses 26 mirrored groups and is implemented in `core/polyhedral.py`, requiring SciPy. Volumes and quadrature use tetrahedron fans of the actual faces.

For structured grids, `refine_uniform()` doubles the intervals in each direction. For Voronoi grids, increase `n_points` and regenerate the mesh. Even with the same seed these are nonnested grids, without parent–child maps or solution transfer. Local adaptive refinement is not implemented. `get_mesh_size()` returns the maximum cell diameter; $N^{-1/d}$ is only a typical scale, not its exact value.

4.1.3 Facets

Codimension-one cell faces (facets) are the basis of boundary detection: in 1D a facet is an endpoint, in 2D an edge, and in 3D a planar polygonal face. `mesh.facets` returns all facets, `mesh.boundary_facets` returns those owned by exactly one cell, and their vertex set is where the Dirichlet boundary lives.

Property 4.1. For a two-dimensional polygonal mesh, facets are exactly the edges of the polygons; for hexahedral meshes they are expanded from a fixed table of six faces; for general three-dimensional polyhedra a face table can be supplied explicitly through `Mesh(..., faces=...)`.

Caution

The `mesh.facets` described above is a "**vertex set used for boundary detection**"; high-order VEM also shares edge points and face moments through its own entity map. Boundary values include normalized face moments. An $H(\mathrm{div})$-conforming mixed element (RT0), however, additionally needs to know "**which cells share a given face**" and "toward which cell the global normal of that face points". The library therefore provides a second layer of facet topology: `core/facets.py`. The next subsection shows how to use it.

4.1.4 Facet Topology and Orientation: `core/facets.py`

The RT0 degrees of freedom are **facet normal fluxes** $\Phi_f = \int_f q \cdot n_f \, \mathrm{d}s$, and two neighbouring cells must share the same unknown. This requires an explicit global numbering and orientation of the facets. The library makes this a first-class citizen of the `core` layer, `FacetTopology` (`mesh.facet_topology`, built lazily and cached):

```
from MosicaFE import Mesh

mesh = Mesh.box(nx=2, ny=2, nz=2, element_type="tet")
t = mesh.facet_topology
```

Table 4.2: Main attributes of `FacetTopology` (`t = mesh.facet_topology`)

Attribute / method	Meaning
`t.facets`	`(n_facets,d)`: **sorted** vertex list of each facet (the global unique key)
`t.cell_facets`	`(n_cells,d+1)`: `[c,i]` is the global index of the i-th local facet of cell c
`t.signs`	± 1: whether the global normal points out of that cell
`t.normals`	unnormalized global normals (magnitude = facet measure)
`t.counts`	number of adjacent cells: 2 for interior facets, 1 for boundary facets
`t.boundary_facets()`	array of boundary facet indices (`t.n_boundary` is the count)

```
5
6  print(t.summary())           # cells=48, facets=120, interior=72, boundary=48
7  print(t.cell_facets[0])      # global indices of the 4 local facets of cell 0
8  print(t.signs[0])            # outward normal signs (+1 / -1)
9
10 # the two signs of an interior facet are necessarily opposite: this is
11 # H(div) conformity in its discrete form
12 c0, c1 = t.facet_cells[0]
13 print(t.signs[c0][list(t.cell_facets[c0]).index(0)] *
14       t.signs[c1][list(t.cell_facets[c1]).index(0)])   # -1
```

Listing 4.2: Facet topology and orientation

Key Point

`build_facet_topology` rejects two kinds of input that would "silently give wrong results": non-simplicial meshes (quadrilaterals / polygons / hexahedra), and meshes with a facet shared by more than two cells (non-conforming partitions). This is the technical reason why RT0 only accepts triangular and tetrahedral meshes.

4.2 Mesh Generation

All meshes are created by class methods of `Mesh`, and all of them use a **counterclockwise (2D) / positive-volume (3D)** orientation.

```
1  from MosicaFE import Mesh
2
3  m1 = Mesh.interval(a=0.0, b=1.0, nx=20)                # 1D interval
4
5  m2 = Mesh.rectangle(nx=8, ny=8)                        # triangles
6  m3 = Mesh.rectangle(nx=8, ny=8, element_type="quad")   # quadrilaterals
7
8  m4 = Mesh.box(nx=4, ny=4, nz=4)                        # tets (Kuhn split)
9  m5 = Mesh.box(nx=4, ny=4, nz=4, element_type="hex")    # hexahedra
10 m6 = Mesh.box(nx=3, ny=3, nz=3, pattern="center")      # 12 tets per cube
11
12 m7 = Mesh.pentagon(nx=8, ny=8)                         # pentagon mesh
```

```
m8 = Mesh.voronoi_polygon(n_points=100, seed=42)       # Voronoi polygons

for m in (m1, m2, m3, m4, m5, m7, m8):
    print(m.summary())
```

Listing 4.3: Generating the various mesh types

Table 4.3: Mesh factory quick reference

Factory	Domain	Cells
`Mesh.interval(a,b,nx)`	$[a, b]$	segments
`Mesh.rectangle(...,"triangle")`	rectangle	triangles
`Mesh.rectangle(...,"quad")`	rectangle	quadrilaterals
`Mesh.box(...,"tet")`	box	tetrahedra
`Mesh.box(...,"hex")`	box	hexahedra
`Mesh.pentagon(nx,ny)`	unit square	pentagons
`Mesh.voronoi_polygon(...)`	unit square	Voronoi polygons
`Mesh.voronoi_polyhedron(...)`	unit cube	Voronoi polyhedra
`Mesh.from_arrays(...)`	user-defined	inferred automatically

Remark 4.1 (Kuhn subdivision)**.** By default `Mesh.box(...,element_type="tet")` splits every small cube into 6 tetrahedra **sharing the main diagonal** (Kuhn subdivision). The triangulations of the common face of two neighbouring cubes then agree, so the mesh is conforming. `pattern="center"` additionally inserts the cube centre and splits into 12 tetrahedra — the mesh used by the original 3D VEM scripts.

Remark 4.2 (Voronoi meshes)**.** When a Voronoi mesh is generated, mirror points are added in 8 directions outside the domain so that the Voronoi cells at the boundary close, after which the subdivision is clipped back to the domain. After clipping and sorting, coincident neighbouring vertices are removed automatically, which avoids degenerate cells (a polygon with repeated vertices would produce zero-area triangles in the fan subdivision).

4.3 Mesh Properties and Geometric Quantities

```
mesh = Mesh.rectangle(nx=8, ny=8)

print(mesh.dim, mesh.n_nodes, mesh.n_cells)      # 2 81 128
print(mesh.get_mesh_size())                      # max cell diameter
print(mesh.cell_measures())                      # array of areas

v = mesh.get_cell_vertices(0)                    # (k, dim) coordinates
h = mesh.get_cell_diameter(0)
c = mesh.get_cell_centroid(0)
print(mesh.boundary_nodes[:5])
```

Listing 4.4: Reading mesh information

4.4 Mesh Refinement

```
coarse = Mesh.rectangle(nx=4, ny=4)
fine = coarse.refine_uniform()
print(coarse.n_cells, fine.n_cells)     # 32 128
```

Listing 4.5: Uniform refinement

`refine_uniform()` is available for structured meshes only (`interval` / `rectangle` / `box` / `pentagon`): it doubles the number of cells in every direction and regenerates the mesh, so the number of triangles is multiplied by 4 and the number of hexahedra by 8. For unstructured meshes such as Voronoi meshes it raises a `ValueError`.

4.5 Reference Elements

Table 4.4: Reference element conventions

Cell type	Reference element
`interval`	$[0,1]$
`triangle`	$\{\xi \geq 0, \eta \geq 0, \xi + \eta \leq 1\}$
`quad`	$[0,1]^2$, vertices ordered $(0,0),(1,0),(1,1),(0,1)$
`tet`	standard tetrahedron, vertices $(0,0,0),(1,0,0),(0,1,0),(0,0,1)$
`hex`	$[0,1]^3$: counterclockwise 4 vertices on $z=0$, then the matching 4 on $z=1$

Simplex cells use the affine map $x = v_0 + J_K \hat{x}$; quadrilaterals and hexahedra use bilinear / trilinear maps on $[0,1]^d$, whose Jacobian matrix depends on the reference point $\hat{x}$ and must therefore be evaluated at every quadrature point.

```
import numpy as np
from MosicaFE.core.utils import (
    compute_jacobian, transform_gradient, apply_affine_map,
    polygon_area, polyhedron_volume,
)

quad = np.array([[0,0],[2,0],[2,2],[0,2]], float)
J, detJ = compute_jacobian(quad, np.array([0.5, 0.5]))
print(J, detJ)                      # [[2,0],[0,2]] 4.0
print(polygon_area(quad))           # 4.0
print(apply_affine_map([[0.5,0.5]], quad))   # [[1, 1]]
```

Listing 4.6: Geometric tools

Caution

The Jacobian convention. `compute_jacobian` returns the **same convention** of Jacobian for both families of cells: columns are partial derivatives of physical coordinates with respect to reference coordinates,

$$J_{ab} = \frac{\partial x_a}{\partial \xi_b}, \qquad J = \left[\ \partial x/\partial \xi_1 \ \ \cdots \ \ \partial x/\partial \xi_d \ \right], \tag{4.1}$$

and only with this convention are the gradient transformation `transform_gradient(dN, J)` (which uses $J^{-\top}$ internally) and the Newton inverse map of quadrilaterals / hexahedra (which uses J^{-1}) correct. **Do not write $dN^{\top}V$ when extending the code** — that is the transposed Jacobian. For axis-aligned rectangles and boxes J is diagonal and symmetric, so the error is completely hidden; but as soon as a mesh is **rotated or distorted** ($J \neq J^{\top}$) the gradients, the stiffness matrix and the interpolation are silently wrong. This library fixed exactly this issue on 2026-09-21 (bug number 7); the regression test is `tests/test_tensor_geometry.py`, which checks J against finite differences and verifies that the errors are identical bit by bit under rotations of 0°/15°/30°/45°.

4.6 Quadrature Rules

```python
from MosicaFE.core.quadrature import QuadratureRule, cell_quadrature

quad = QuadratureRule.get_quadrature("triangle", order=3)
print(quad.n_points, quad.weights.sum())     # 4 0.5

f = lambda xi: xi[0]**2 + xi[1]**2
value = sum(quad.weights[q] * f(quad.points[q]) for q in range(quad.n_points))
print(value)                                  # 1/6

# map to a physical cell (fan subdivision for polygons / polyhedra is automatic)
mesh = Mesh.voronoi_polygon(n_points=20, seed=1)
pts, w = cell_quadrature(mesh, 0, order=3)
print(w.sum(), mesh.get_cell_measure(0))
```

Listing 4.7: Using quadrature rules

Table 4.5: Available quadrature rules (`order` is the degree of the polynomial to be integrated exactly)

Cell	Rule	Remarks
`interval`	Gauss–Legendre	$n = \lceil (order+1)/2 \rceil$ points, normalized to $\int_0^1$
`triangle`	barycentre / 3-point / 4-point / 6-point	exact for degree 1/2/3/4 respectively
`quad`	tensor-product Gauss	n points per direction
`tet`	1-point / 4-point / 5-point	exact for degree 1/2/3 respectively
`hex`	tensor-product Gauss	n points per direction

Caution

For tensor-product cells the quadrature order has to be understood **per direction**. The integrand of the `Q1` stiffness matrix is quadratic in each direction, so at least 2 Gauss points per direction are needed — MosicaFE automatically rounds the order up to `max(2, 2*p)`, which the user normally does not have to specify. Earlier versions used one point too few here, which made the computed convergence order of 3D hexahedral elements negative.

Chapter 5

Discrete Spaces

5.1 Space Abstraction

All spaces derive from `BaseFESpace` and expose a **matrix-level interface** to the layers above:

Table 5.1: Key methods of `BaseFESpace`

Method	Meaning
`stiffness_matrix()`	$A_{ij} = \int \nabla\varphi_j \cdot \nabla\varphi_i$
`mass_matrix()`	$M_{ij} = \int \varphi_j \varphi_i$
`load_vector(f)`	$b_i = \int f\varphi_i$
`boundary_dofs()`	indices of the Dirichlet degrees of freedom
`interpolate(f)`	interpolate a function onto the degrees of freedom
`evaluate(cell,u,pts)`	reconstruct a function from the degrees of freedom inside a cell and evaluate it
`evaluate_gradient(cell,u,pts)`	as above, returning the gradient
`nodal_values(u)`	map a degree-of-freedom vector back to the mesh nodes (for visualization)
`get_cell_dofs(i)`	global indices of the degrees of freedom of cell i
`get_dof_coordinates(k)`	physical coordinates of degree of freedom k
`eval_basis(xi)` / `eval_basis_grad(xi)`	basis functions on the reference element (explicit-basis spaces only)

Key Point

This set of interfaces is the reason why "a new PDE takes a dozen lines": the physics layer only ever forms linear combinations of matrices and handles boundary conditions; it never has to know about the cell loop.

5.2 Lagrange Elements (Classical Finite Elements)

5.2.1 P1 Triangles

On the reference triangle, with barycentric coordinates $\lambda_0 = 1 - \xi - \eta$, $\lambda_1 = \xi$, $\lambda_2 = \eta$:

$$\hat{\varphi}_0 = 1 - \xi - \eta, \qquad \hat{\varphi}_1 = \xi, \qquad \hat{\varphi}_2 = \eta. \tag{5.1}$$

5.2.2 P2 Triangles

$$\begin{aligned} &\hat{\varphi}_0 = \lambda_0(2\lambda_0 - 1), && \hat{\varphi}_1 = \lambda_1(2\lambda_1 - 1), && \hat{\varphi}_2 = \lambda_2(2\lambda_2 - 1), \\ &\hat{\varphi}_3 = 4\lambda_0\lambda_1 \text{ (edge 0-1)}, && \hat{\varphi}_4 = 4\lambda_1\lambda_2 \text{ (edge 1-2)}, && \hat{\varphi}_5 = 4\lambda_2\lambda_0 \text{ (edge 2-0)}. \end{aligned} \tag{5.2}$$

5.2.3 Tensor-Product Elements

Quadrilaterals and hexahedra use tensor products of the 1D Lagrange basis. For Q2, for instance, with the 1D nodes $\{0, \frac{1}{2}, 1\}$:

$$N_0(t) = 2t^2 - 3t + 1, \qquad N_1(t) = -4t^2 + 4t, \qquad N_2(t) = 2t^2 - t, \tag{5.3}$$

the cell basis functions are $N_{ijk}(\xi, \eta, \zeta) = N_i(\xi)N_j(\eta)N_k(\zeta)$. Since the nodes form a complete tensor grid, $\{N_{ijk}\}$ is automatically a **nodal basis** dual to the nodal values.

5.2.4 Global Numbering of the Degrees of Freedom

MosicaFE uses a "register" that maps geometric entities to global degrees of freedom:

- vertex degrees of freedom: keyed by the global vertex index;
- edge degrees of freedom: keyed by the **ordered pair** of endpoints;
- face degrees of freedom (3D): keyed by the ordered 4-tuple of face vertices;
- cell-interior degrees of freedom: keyed by the cell index.

Neighbouring cells therefore share edge / face degrees of freedom automatically, and the global space is H^1-conforming.

```python
from MosicaFE import Mesh, FESpace

mesh = Mesh.rectangle(nx=8, ny=8)

V1 = FESpace.lagrange(mesh, degree=1)
V2 = FESpace.lagrange(mesh, degree=2)
print(V1.n_dofs, V2.n_dofs)      # 81 289

print(V2.get_cell_dofs(0))       # local -> global DOF map
print(V2.get_dof_coordinates(0))
print(V2.boundary_dofs()[:6])
```

Listing 5.1: Creating spaces and inspecting the degrees of freedom

5.3 Virtual Element Spaces

```python
import numpy as np
from MosicaFE import Mesh, FESpace

mesh = Mesh.voronoi_polygon(n_points=100, seed=42)
k = 3
V = FESpace.vem(mesh, degree=k)
# Equivalent: FESpace.create(mesh, family="vem", degree=k)
print(V.n_dofs, mesh.n_nodes)
A = V.stiffness_matrix()
```

```
M = V.mass_matrix()
b = V.load_vector(lambda x: 1.0)
one = V.interpolate(lambda x: 1.0)
print(np.max(np.abs(M @ one - b)))
```

Listing 5.2: Arbitrary-order VEM space and constant-function check

`degree` must be a positive integer and defaults to 1; `element="VEM3"` is another spelling. At $k \geq 2$, `u` includes moments, so the number of unknowns usually exceeds the number of vertices. `V.interpolate` evaluates every functional. The moment entries of a constant are not all 1: check $M\mathbf{1}_{\text{dof}} = b$ as above, rather than matrix row sums. `V.nodal_values(u)` returns the actual vertex values and is suitable for reporting their range.

Property 5.1. On admissible meshes with adequate quadrature, the default local stiffness is symmetric positive semi-definite, contains constants in its kernel, and is consistent on all of $\mathcal{P}_k$. Appropriate Dirichlet conditions allow direct or SPD iterative solution. Polynomial consistency tests currently cover $k \leq 5$. There is no artificial degree cap; conditioning worsens at high order, which is not a claim that every arbitrarily high degree has been validated.

Table 5.2: Stabilization options

Value	Meaning
`"trace"`	Default; symmetric stabilization with trace scaling
`"legacy"`	Compatibility alias for trace
`"none"`	Projection term only; diagnostics

5.4 RT0×P0 Mixed Space

```
from MosicaFE import Mesh, FESpace

mesh = Mesh.rectangle(nx=16, ny=16)       # triangle mesh
V = FESpace.rt0(mesh)

print(V.n_flux, V.n_pressure, V.n_dofs)   # facets, cells, sum
print(V.cell_flux_dofs(0))                # local facet -> global flux DOF
print(V.facet_signs(0))                   # outward orientation signs
print(V.local_flux_mass(0))               # exact RT0 mass matrix of cell 0 (3x3)
```

Listing 5.3: Mixed space

5.4.1 The Two Blocks Exposed by the Space Layer

The interface of the RT0 space is **parallel** to that of `LagrangeSpace` and `VEMSpace`; only the "single elliptic operator" is replaced by "the two blocks of a saddle-point problem":

```
from MosicaFE import Mesh, FESpace
from MosicaFE._compat import bmat

mesh = Mesh.rectangle(nx=16, ny=16)
V = FESpace.rt0(mesh)

```

Table 5.3: Matrix primitives of RT0×P0 (combined by the physics layer)

Method	Shape	Meaning
`V.flux_mass_matrix()`	(Φ, Φ)	$M_{ij} = \int_K \varphi_i \cdot \varphi_j$, **closed form, exact**, no quadrature needed
`V.divergence_matrix()`	(P, Φ)	$B_{K,i} = \int_K \nabla \cdot \varphi_i = \sigma_{K,i}$, always ± 1
`V.pressure_load_vector(f)`	$(P,)$	$\int_K f$
`V.interpolate_pressure(f)`	$(P,)$	cell average of f (L^2 projection onto P0)
`V.interpolate_flux(q)`	$(\Phi,)$	$\Phi_f = \int_f q \cdot n_f \, \mathrm{d}s$
`V.evaluate_flux(c,Q,pts)`	(n_q, d)	reconstruct $q_h = \sum_i F_i \varphi_i$ in a cell
`V.locate_cell(x)`	int	locate the cell containing a point via barycentric coordinates

```
M = V.flux_mass_matrix()           # (Phi,Phi)
B = V.divergence_matrix()          # (P,Phi)
f = V.pressure_load_vector(src)    # (P,)

A = bmat([[M, -B.T], [B, None]])  # [[M, -B^T], [B, 0]]
```

Listing 5.4: Using the two blocks directly

Caution

The RT0 space **does not provide** `stiffness_matrix()` / `mass_matrix()` / `load_vector()`, nor `interpolate()`: it corresponds to a saddle-point problem and carries two fields at once, so there is no "single operator" and no "single interpolation". Calling those methods raises a `NotImplementedError` whose message points to the method to use instead (for example `mass_matrix()` → `flux_mass_matrix()`, `load_vector()` → `pressure_load_vector()`). In day-to-day use, combine the space with `MixedPoissonProblem`.

5.5 The Space Factory `FESpace`

```
from MosicaFE import Mesh, FESpace

mesh = Mesh.rectangle(nx=16, ny=16)

V_fem = FESpace.lagrange(mesh, degree=2)     # P2
V_vem = FESpace.vem(mesh, degree=3)          # VEM k=3
V_rt0 = FESpace.rt0(mesh)                    # RT0 x P0

# by name (used by solve_poisson(element=...))
print(FESpace.available())
V_any = FESpace.create(mesh, family="vem", degree=3)
```

Listing 5.5: The three space families

Table 5.4: String names of the "elements" (used by `solve_poisson(element=...)`)

Name	Space family	Available meshes
`"P1"` / `"P2"`	Lagrange	triangles, tetrahedra
`"Q1"` / `"Q2"`	Lagrange (tensor product)	quadrilaterals, hexahedra
`"VEM"` / `"VEM3"`	VEM, $k \geq 1$	admissible polygons / polyhedra
`"RT0"` / `"RT0-P0"` / `"MIXED"`	RT0×P0	triangles, tetrahedra

Chapter 6

Physics Problems and Assembly

6.1 The Physics Problem Interface

All PDEs derive from `BasePhysics` and only have to implement one method:

```python
class BasePhysics(ABC):
    def __init__(self, space, f=None, g=None, **kwargs):
        self.space = space
        self.mesh = space.mesh
        self.n_dofs = space.n_dofs
        self.f = f or (lambda x: 0.0)      # source term
        self.g = g or (lambda x: 0.0)      # Dirichlet data
        self.boundary_dofs = space.boundary_dofs()

    @abstractmethod
    def assemble(self):
        """return (A, b) with boundary conditions applied"""

    def solve(self, method="auto", **kwargs):
        A, b = self.assemble()
        return Solver.solve(A, b, method=method, **kwargs)
```

Listing 6.1: The `BasePhysics` interface

6.2 The Poisson Problem

```python
@register_problem("poisson")
class PoissonProblem(BasePhysics):
    def __init__(self, space, f=None, g=None, kappa=1.0, **kwargs):
        super().__init__(space, f=f, g=g, **kwargs)
        self.kappa = kappa

    def assemble(self):
        A = self.space.stiffness_matrix()
        if float(self.kappa) != 1.0:
            A = float(self.kappa) * A
        b = self.space.load_vector(self.f)
        A, b = self.apply_boundary_conditions(A, b)
        return A, b
```

Listing 6.2: Assembly of the Poisson problem (complete implementation)

Key Point

The whole implementation is 8 lines long, and it holds **simultaneously for Lagrange elements and for virtual elements**. This is the payoff of pushing the "cell loop" down into the `MosicaFE.spaces` layer.

6.3 The Helmholtz Problem

The Helmholtz problem (3.2) differs from the Poisson problem by a single term, and correspondingly its implementation differs by a single line: the stiffness matrix is replaced by the *indefinite* combination $\kappa K - k^2 M$.

```python
@register_problem("helmholtz")
class HelmholtzProblem(BasePhysics):
    """-div(kappa grad u) - k**2 u = f"""

    def __init__(self, space, f=None, g=None, k=1.0, kappa=1.0, **kwargs):
        super().__init__(space, f=f, g=g, **kwargs)
        self.k = float(k)              # constant wavenumber
        self.kappa = float(kappa)      # constant diffusion coefficient

    def assemble(self):
        A = self.space.stiffness_matrix() - self.k**2 * self.space.mass_matrix()
        if self.kappa != 1.0:
            A = self.kappa * A
        b = self.space.load_vector(self.f)
        A, b = self.apply_boundary_conditions(A, b)
        return A, b
```

Listing 6.3: Assembly of the Helmholtz problem (complete implementation)

The class is a subclass of `BasePhysics`, so it inherits `solve`, `compute_error`/`compute_errors` and everything else unchanged; the only thing that had to be supplied is the combination of the two matrices that the space already computed. Consequently the Helmholtz problem accepts **every H1-conforming space**, exactly as the Poisson problem does:

```python
import numpy as np
from MosicaFE import FESpace, HelmholtzProblem, Mesh, solve_helmholtz

k = 1.0
exact = lambda x: np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
source = lambda x: (2*np.pi**2 - k**2) * exact(x)     # manufactured solution

mesh = Mesh.rectangle(nx=32, ny=32)

u = HelmholtzProblem(FESpace.lagrange(mesh, 2), f=source, g=lambda x: 0.0,
                     k=k).solve(method="direct")
u = HelmholtzProblem(FESpace.vem(mesh), f=source, g=lambda x: 0.0,
                     k=k).solve(method="direct")

u = solve_helmholtz(mesh, element="P2", f=source, k=k)   # one-line helper
u = run("helmholtz", mesh, element="VEM", f=source, k=k) # by registry name
```

Listing 6.4: The same equation on any element family

Key Point

Three practical points, all of them consequences of (3.2) rather than of the library.

(1) **Choose the solver by the shift, not by the element.** For $k^2 < \lambda_{1,h}$ the matrix $\kappa K - k^2 M$ is symmetric positive definite, so `"auto"`, `"direct"` and `"cg"` are all usable. Above the first eigenvalue it is symmetric *indefinite*: `"direct"` and `"minres"` still work, CG does not.

(2) **Report kh next to the error.** The a priori estimate (3.20) is an $h \to 0$ statement; for a wave problem the pre-asymptotic (pollution) regime can be wide, so a convergence table is only meaningful together with the resolution indicator kh.

(3) **Constant coefficients only.** The library accepts a real constant k and a real constant κ and rejects anything else with an explicit error message rather than assembling a matrix silently. The variant $-\Delta u + k^2 u = f$ (the screened Poisson equation) is the built-in `ReactionDiffusionProblem` with `reaction_coeff=k**2`; complex wavenumbers (absorbing media) would require a complex-valued pipeline and are outside the current scope.

Table 6.1 collects the measured verification data for the constant-wavenumber case $k = 1$; the discrete first eigenvalue was obtained as the smallest eigenvalue of the generalized problem $Kv = \lambda Mv$ restricted to the interior degrees of freedom.

Table 6.1: Helmholtz verification ($k = 1$, $u = \sin(\pi x)\sin(\pi y)$, constant wavenumber)

Space / mesh	L^2 order	H^1 order	$k^2/\lambda_{1,h}$	kh range
P1 / triangles	1.98	0.99	0.0505	$0.25 \to 0.031$
P2 / triangles	3.01	1.99	0.0507	$0.25 \to 0.031$
Q1 / quadrilaterals	2.00	1.00	0.0507	$0.25 \to 0.031$
VEM $k = 1$ / triangles	1.98	0.99	0.0505	$0.25 \to 0.031$

The measured orders coincide with the Poisson ones, which is the point: in the low-frequency regime the Helmholtz problem is *not* harder than the Poisson problem, and the same elements, solvers and postprocessing apply. The discrete eigenvalues $\lambda_{1,h} = 19.7868$ (P1, VEM) and 19.7394 (P2) bracket the exact value $2\pi^2 = 19.7392$ from above, as the Galerkin eigenvalue theory predicts.

6.4 Boundary Conditions

6.4.1 Imposing Dirichlet Conditions Correctly

To impose $u_j = g_j$ on the boundary degrees of freedom j, the standard approach is "eliminate the row and the column, and correct the right-hand side":

(1) move the contribution of the boundary columns to the right-hand side:

$$b_i \leftarrow b_i - \sum_{j \in \mathcal{D}} A_{ij} g_j, \qquad \forall i \notin \mathcal{D}; \tag{6.1}$$

(2) delete all rows and columns involving $\mathcal{D}$;

(3) set $A_{jj} = 1$ and $b_j = g_j$ for $j \in \mathcal{D}$.

Caution

Step (1) "looks" dispensable for homogeneous conditions $g = 0$, which is why many demo programs leave it out; as soon as the boundary data are nonzero the solution is wrong. MosicaFE guards against a regression of this kind with a simple but sensitive test — the **inhomogeneous linear patch test**:

$$u(x) = 1 + 2x_1 - \tfrac{1}{2}x_2, \qquad f \equiv 0, \qquad g = u|_{\partial\Omega}, \tag{6.2}$$

where the exact solution belongs to the discrete space, so the numerical solution must be **exact** (error $\sim 10^{-15}$). Before the fix, the measured error was 2.875.

6.4.2 Other Boundary Conditions

- **Neumann conditions**: add $\int_{\Gamma_N} h\varphi_i \, ds$ to the result of `load_vector`; this requires a quadrature rule on the boundary facets;
- **RT0 mixed form:** $p = 0$ is a natural boundary condition, so no degree of freedom has to be eliminated (which is also why mixed formulations are more "natural" for flux problems);
- inhomogeneous data $p = g \neq 0$ in the mixed form require an additional boundary flux term; the current version reports this explicitly instead of returning a wrong result.

6.5 Assembly Algorithm

Algorithm 1 Global assembly (`BaseFESpace._assemble` in MosicaFE)

Require: space V, cell-local matrix generator `local_matrix(cell)`

initialize the triplet lists `rows,cols,vals`
for each cell $c = 0, \ldots, n_{\text{cells}} - 1$ **do**
  $\mathcal{D}_c \leftarrow$`get_cell_dofs(c)`
  $K_c \leftarrow$`local_matrix(c)`
  append the flattened row / column indices of $\mathcal{D}_c$ and K_c to the triplet lists
end for
build the matrix from the COO triplets (repeated indices are summed)
if symmetry is required **then**
  $A \leftarrow \frac{1}{2}(A + A^\top)$
end if
return A

Caution

The right-hand side must be assembled with "per-entry accumulation" semantics (`numpy`'s `np.add.at`). If one writes `b[dofs] += local` directly, the buffered assignment of `numpy` silently drops accumulations whenever `dofs` contains repeated indices (for instance when the mesh has duplicate vertices), while the COO assembly of the sparse matrix sums them correctly; this inconsistency produces errors that are very hard to notice.

6.6 Using the Poisson Problem

```python
import numpy as np
from MosicaFE import Mesh, FESpace, PoissonProblem

def source(x):
    return 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

mesh = Mesh.rectangle(nx=32, ny=32)
V = FESpace.lagrange(mesh, degree=2)

problem = PoissonProblem(V, f=source, g=lambda x: 0.0, kappa=1.0)

A, b = problem.assemble()
print("system size:", A.shape, "non-zeros:", A.nnz)

u = problem.solve(method="direct")
print("solution size:", u.size)
```

Listing 6.5: Assembling and solving

6.7 The PDE Registry

```python
from MosicaFE import FESpace, Mesh, available_problems, create_problem, run

print(available_problems())        # ['mixed_poisson', 'poisson', ...]

mesh = Mesh.rectangle(nx=32, ny=32)
problem = create_problem("poisson", FESpace.vem(mesh),
                         f=source, g=lambda x: 0.0)
u = problem.solve()

u2 = run("poisson", mesh, element="VEM", f=source, g=lambda x: 0.0)
```

Listing 6.6: Creating and solving equations by name

Chapter 7

Linear Solvers

7.1 The Solver Interface

```python
import numpy as np, time
from MosicaFE import FESpace, Mesh, PoissonProblem, Solver

mesh = Mesh.rectangle(nx=32, ny=32)
problem = PoissonProblem(FESpace.lagrange(mesh, 2), f=source,
                         g=lambda x: 0.0)
A, b = problem.assemble()

t0 = time.time(); u_direct = Solver.direct(A, b); t1 = time.time()
u_cg    = Solver.conjugate_gradient(A, b, tol=1e-10)
u_gmres = Solver.gmres(A, b, tol=1e-10, restart=30)
u_bicg  = Solver.bicgstab(A, b, tol=1e-10)

print("direct %.3f s, diff vs CG: %.3e"
      % (t1 - t0, np.linalg.norm(u_direct - u_cg)))
```

Listing 7.1: The `Solver` interface

The problem object can also choose for itself:

```python
u = problem.solve(method="auto")   # auto/direct/cg/gmres/bicgstab/minres
```

7.2 Direct Methods

- sparse matrices: `scipy.sparse.linalg.spsolve` (sparse LU / Cholesky internally);
- dense matrices: `numpy.linalg.solve`;
- complexity: about $O(n^{1.5})$ in 2D and $O(n^2)$ in 3D.

7.3 Iterative Methods

Caution

CG requires a symmetric positive definite matrix after boundary conditions. VEM `trace` and `legacy` now use the same symmetric stabilization. The diagnostic `none` option does not generally control the projection kernel.
For saddle-point systems use a direct method or MINRES.

Table 7.1: Available Krylov solvers

Method	Applicable matrices	Remarks
`conjugate_gradient`	symmetric positive definite	$O(\text{nnz})$ per step, iteration count $O(\sqrt{\kappa})$
`gmres`	general (nonsymmetric)	with restarts, storage $O(\text{restart} \times n)$
`bicgstab`	general	two matrix-vector products per step, low memory
`minres`	symmetric indefinite	e.g. the RT0 saddle-point system

7.4 Solver Selection Guide

Table 7.2: Solver selection recommendations

Problem size	Matrix type	Recommended solver	Remarks
$< 2 \times 10^4$ DOFs	any	direct	exact, no tuning required
$> 2 \times 10^4$	symmetric positive definite	CG	preconditioning recommended
$> 2 \times 10^4$	symmetric indefinite	MINRES or direct	saddle-point problems, and Helmholtz with $k^2 > \lambda_1$
$> 2 \times 10^4$	nonsymmetric	GMRES / BiCGSTAB	needs a restart parameter

```python
from MosicaFE import Solver

print(Solver.recommend(1000, symmetric=True))      # 'direct'
print(Solver.recommend(10**6, symmetric=True))     # 'cg'
print(Solver.recommend(10**6, symmetric=False))    # 'gmres'
```

Listing 7.2: Automatic selection

Chapter 8

Error Analysis and Convergence

8.1 Error Norms

$$\|e\|_{L^2} = \left(\int_\Omega |u - u_h|^2 \,\mathrm{d}x\right)^{1/2}, \qquad \|e\|_{L^2}^2 \approx \sum_K \sum_q w_q \, |u(x_q) - u_h(x_q)|^2 |\det J_K|, \tag{8.1}$$

$$|e|_{H^1} = \left(\int_\Omega |\nabla u - \nabla u_h|^2 \,\mathrm{d}x\right)^{1/2}, \qquad \|e\|_{L^\infty} = \max_{x \in \Omega} |u(x) - u_h(x)|. \tag{8.2}$$

```python
import numpy as np

def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def exact_grad(x):
    return np.array([np.pi*np.cos(np.pi*x[0])*np.sin(np.pi*x[1]),
                     np.pi*np.sin(np.pi*x[0])*np.cos(np.pi*x[1])])

u = problem.solve()
l2   = problem.compute_error(u, exact, "L2")
linf = problem.compute_error(u, exact, "Linf")
h1   = problem.compute_error(u, exact, "H1", exact_grad=exact_grad)

print("%.6e %.6e %.6e" % (l2, h1, linf))

norms = problem.compute_errors(u, exact, exact_grad)
```

Listing 8.1: Computing the three norms

Remark 8.1. `VEMSpace.evaluate` uses $\Pi_k^0 u_h$; `evaluate_gradient` uses $\nabla \Pi_k^\nabla u_h$. The reported errors are computable projection errors, not exact errors of the implicit virtual function. `H1` is a gradient seminorm error and `Linf` is a sampled maximum. Moment entries in the solution vector are not point values and must not be used to report a function range.

8.2 Convergence Study Framework

```python
from MosicaFE import FESpace, Mesh, PoissonProblem, convergence_study
from MosicaFE.postprocess import estimate_convergence_rate

def mesh_factory(level):
```

```
    n = 4 * 2**level
    return Mesh.rectangle(nx=n, ny=n)

def problem_factory(mesh):
    V = FESpace.lagrange(mesh, degree=2)
    return PoissonProblem(V, f=source, g=lambda x: 0.0)

h, errors = convergence_study(mesh_factory, problem_factory,
                              exact, exact_grad,
                              refinement_levels=5, plot=True,
                              filename="conv_p2.png")

print(estimate_convergence_rate(h, errors["L2"], use_last=3))   # ~3.0
```

Listing 8.2: Automatic convergence study

8.3 Convergence Rate Estimation

From the errors on two successive levels,

$$\text{rate} = \frac{\log(e_1/e_2)}{\log(h_1/h_2)}. \tag{8.3}$$

A more robust choice is a least-squares fit of $\log e$ against $\log h$ (which is how `estimate_convergence_rate(h,errors)` is implemented); the option `use_last=k` restricts the fit to the last k levels in order to approximate the asymptotic order.

8.4 Measured Verification Results

The table retains the existing FEM/RT0 validation records. Traceable results for the updated arbitrary-order VEM appear in Section 10.3.1.

Table 8.1: Measured convergence orders (manufactured solution, successively refined meshes)

Element	Mesh	measured L^2	measured H^1	theory (L^2/H^1)
P1	triangles	1.98	0.99	2 / 1
P2	triangles	3.00	1.99	3 / 2
Q1	quadrilaterals	2.00	1.00	2 / 1
Q2	quadrilaterals	3.00	2.00	3 / 2
P1	tetrahedra	1.95	0.98	2 / 1
Q1	hexahedra	2.01	1.00	2 / 1
RT0×P0	triangles (pressure)	1.00	—	1
RT0×P0	triangles (flux)	1.00	—	1

To reproduce the table:

```
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_checks.py
```

Remark 8.2. The same script also prints the Helmholtz table for the constant-wavenumber case $k = 1$ (Table 6.1): the measured orders are 1.98/0.99 (P1), 3.01/1.99 (P2), 2.00/1.00 (Q1) and 1.98/0.99 (VEM), i.e. identical to the Poisson orders of Table 8.1. The output also

contains the resolution indicator kh together with $\lambda_{1,h}$, the discrete first Dirichlet eigenvalue, so that the table can be checked against the well-posedness condition $k^2 < \lambda_1$ of Chapter 6.3.

Chapter 9

Visualization

9.1 Overview of the Plotting Tools

Table 9.1: Plotting functions in `MosicaFE.postprocess`

Function	Purpose
`plot_solution(u,mesh,V,...)`	plot the solution. In 1D a curve; in 2D filled contours or a 3D surface; in 3D a slice $z = \text{const}$. Polygonal meshes are triangulated by a fan from the cell centroids
`plot_mesh(mesh,...)`	plot the mesh (in 3D a mid-plane slice)
`plot_error(u,mesh,space, exact,...)`	plot the pointwise error $\lvert u_h - u \rvert$
`plot_convergence(h,errors, ...)`	log–log convergence curves with reference slopes $O(h^k)$ overlaid
`plot_pressure_2d(mesh,P, ...)`	piecewise constant RT0 pressure
`plot_flux_2d(mesh,Q,...)`	flux arrow plot
`compare_solutions(entries, ...)`	side-by-side comparison of several solutions

Common keyword arguments: `title=`, `filename=` (saves a PNG), `show=` (opens a window), `dpi=`, and `ax=` (draw into an existing axes object, e.g. to build a panel figure).

9.2 Plotting Solutions

```python
from MosicaFE import Mesh, FESpace, PoissonProblem
from MosicaFE.postprocess import plot_solution, plot_mesh, plot_error

# --- 2D contour ---
mesh = Mesh.rectangle(nx=32, ny=32)
V = FESpace.lagrange(mesh, 2)
u = PoissonProblem(V, f=source, g=lambda x: 0.0).solve()
plot_solution(u, mesh, V, title="FEM P2", filename="sol2d.png")

# --- 2D surface ---
plot_solution(u, mesh, V, mode="surface", filename="sol_surface.png")
```

```

# --- polygon mesh + VEM solution ---
mp = Mesh.voronoi_polygon(n_points=200, seed=3)
Vv = FESpace.vem(mp)
uv = PoissonProblem(Vv, f=source, g=lambda x: 0.0).solve()
plot_solution(uv, mp, Vv, title="VEM on polygons",
              filename="sol_vem.png")
plot_error(uv, mp, Vv, exact, filename="err_vem.png")

# --- 3D slice ---
m3 = Mesh.box(nx=8, ny=8, nz=8)
V3 = FESpace.lagrange(m3, 1)
u3 = PoissonProblem(V3, f=source3d, g=lambda x: 0.0).solve()
plot_solution(u3, m3, V3, z_slice=0.5, filename="sol3d_slice.png")
plot_mesh(m3, filename="mesh3d.png")
```

Listing 9.1: Plotting 2D and 3D solutions

9.3 Plotting Convergence Curves

```
from MosicaFE.postprocess import plot_convergence

plot_convergence(h, errors, filename="convergence.png",
                 reference_rates=(1, 2, 3))
```

Listing 9.2: Convergence curves

Caution

When `matplotlib` is not installed, all plotting functions merely print a notice and return `None` instead of raising an exception, so batch scripts may call them freely.

Chapter 10

Complete Examples

10.1 Example 1: Solving Poisson in Two Lines

```python
import numpy as np
from MosicaFE import Mesh, solve_poisson

source = lambda x: 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

u = solve_poisson(Mesh.rectangle(nx=32, ny=32), element="P2", f=source)
print(u.size)          # 4225
```

Listing 10.1: One-line solution

10.2 Example 2: Changing the Element (One Code, Three Methods)

```python
import numpy as np
from MosicaFE import FESpace, Mesh, MixedPoissonProblem, PoissonProblem

def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def grad(x):
    return np.array([np.pi*np.cos(np.pi*x[0])*np.sin(np.pi*x[1]),
                     np.pi*np.sin(np.pi*x[0])*np.cos(np.pi*x[1])])

def source(x):
    return 2*np.pi**2 * exact(x)

def zero(x):
    return 0.0

mesh = Mesh.rectangle(nx=32, ny=32)

# (a) classic FEM: P2 Lagrange
V = FESpace.lagrange(mesh, degree=2)
p = PoissonProblem(V, f=source, g=zero)
u = p.solve()
print("FEM P2 :", p.compute_errors(u, exact, grad))

```

```python
# (b) VEM: only the space changes
V = FESpace.vem(mesh, degree=3)
p = PoissonProblem(V, f=source, g=zero)
u = p.solve()
print("VEM k=3:", p.compute_errors(u, exact, grad))

# (c) RT0 x P0 mixed: space + problem class
V = FESpace.rt0(mesh)
p = MixedPoissonProblem(V, f=source)
Q, P = p.solve()
print("RT0-P0 :", p.compute_errors((Q, P), exact, lambda x: -grad(x)))
```

Listing 10.2: FEM / VEM / RT0 share the same workflow

The program prints errors for each method. Current fine-grid VEM measurements follow below.

10.3 Example 3: Separate 2D and 3D Arbitrary-Order VEM Programs

The programs retain the FEM reading order: problem data, mesh, element and solver, then visualization. On $(0,1)^d$, use $-\Delta u = f$, $u = \prod_{j=1}^{d} \sin(\pi x_j)$, $f = d\pi^2 u$, and zero boundary data. Save and run the following as separate scripts under `examples`. The repository versions, `examples/vem_poisson.py` and `examples/vem_poisson_3d.py`, additionally save mesh images, errors and JSON data.

```python
import os
import sys
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.dirname(os.path.dirname(HERE)))
from MosicaFE import Mesh, plot_solution, solve_poisson

OUT = os.path.join(HERE, "output", "vem_2d")
os.makedirs(OUT, exist_ok=True)

# Problem data.
def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def source(x):
    return 2*np.pi**2*exact(x)

def main(n_points=1024, degree=2):
    # Mesh.
    mesh = Mesh.voronoi_polygon(n_points=n_points, seed=42)
    # Element, solver, and solve.
    out = solve_poisson(mesh, element="VEM", degree=degree,
                        f=source, g=lambda x: 0.0,
                        solver="direct", return_all=True)
    u, V = out["u"], out["space"]
    # Visualization.
    plot_solution(u, mesh, V, title=f"VEM{degree} (2D)",
                  filename=os.path.join(OUT, f"solution_k{degree}.png"))
    values = V.nodal_values(u)
```

```
    print("DOFs:", u.size, "vertex range:", values.min(), values.max())

if __name__ == "__main__":
    main()
```

Listing 10.3: Standalone 2D main program

```
import os
import sys
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.dirname(os.path.dirname(HERE)))
from MosicaFE import Mesh, plot_solution, solve_poisson

OUT = os.path.join(HERE, "output", "vem_3d")
os.makedirs(OUT, exist_ok=True)

# Problem data.
def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1]) * np.sin(np.pi*x[2])

def source(x):
    return 3*np.pi**2*exact(x)

def main(n_points=192, degree=2):
    # Mesh.
    mesh = Mesh.voronoi_polyhedron(n_points=n_points, seed=42)
    # Element, solver, and solve.
    out = solve_poisson(mesh, element="VEM", degree=degree,
                        f=source, g=lambda x: 0.0,
                        solver="direct", return_all=True)
    u, V = out["u"], out["space"]
    # Visualization.
    plot_solution(u, mesh, V, title=f"VEM{degree} (3D)", z_slice=0.5,
                  filename=os.path.join(OUT, f"solution_k{degree}.png"))
    values = V.nodal_values(u)
    print("DOFs:", u.size, "vertex range:", values.min(), values.max())

if __name__ == "__main__":
    main()
```

Listing 10.4: Standalone 3D main program

Change `degree` to select the order; `main(n_points=...,degree=3)` runs the cubic method. Defaults are 1024 polygons in 2D and 192 polyhedra in 3D, with separate output folders. The 2D display interpolates vertex values. In 3D, `z_slice=0.5` cuts the integration tetrahedra with the actual plane and evaluates the L^2 projection at intersection points.

10.3.1 Fine-Mesh Results and Error Plots

The table uses the saved `examples/output/vem_2d/summary.json` and `examples/output/vem_3d/summary.json`, both with seed 42. The reported errors are $\|u - \Pi_k^0 u_h\|_{L^2}$ and $\|\nabla u - \nabla\Pi_k^\nabla u_h\|_{L^2}$; the H^1 column is a seminorm error.

The dimensions are plotted separately. Each dimension has only two nonnested quadratic grids and one cubic fine-grid result. These show refinement and degree effects, not an established asymptotic rate. Under the usual mesh regularity, solution smoothness, stabilization and

Table 10.1: Saved fine-mesh VEM results in 2D and 3D

d	N	k	DOFs	L^2	H^1
2	256	2	1539	1.952×10^{-4}	1.328×10^{-2}
2	1024	2	6147	2.548×10^{-5}	3.378×10^{-3}
2	1024	3	11268	4.038×10^{-7}	6.825×10^{-5}
3	96	2	2361	7.294×10^{-3}	1.781×10^{-1}
3	192	2	4977	4.015×10^{-3}	1.212×10^{-1}
3	192	3	10527	9.961×10^{-4}	1.919×10^{-2}

quadrature assumptions, the expected orders are $O(h^k)$ in H^1 and $O(h^{k+1})$ in L^2. Conditioning of the high-order monomial basis can affect numerical behavior.

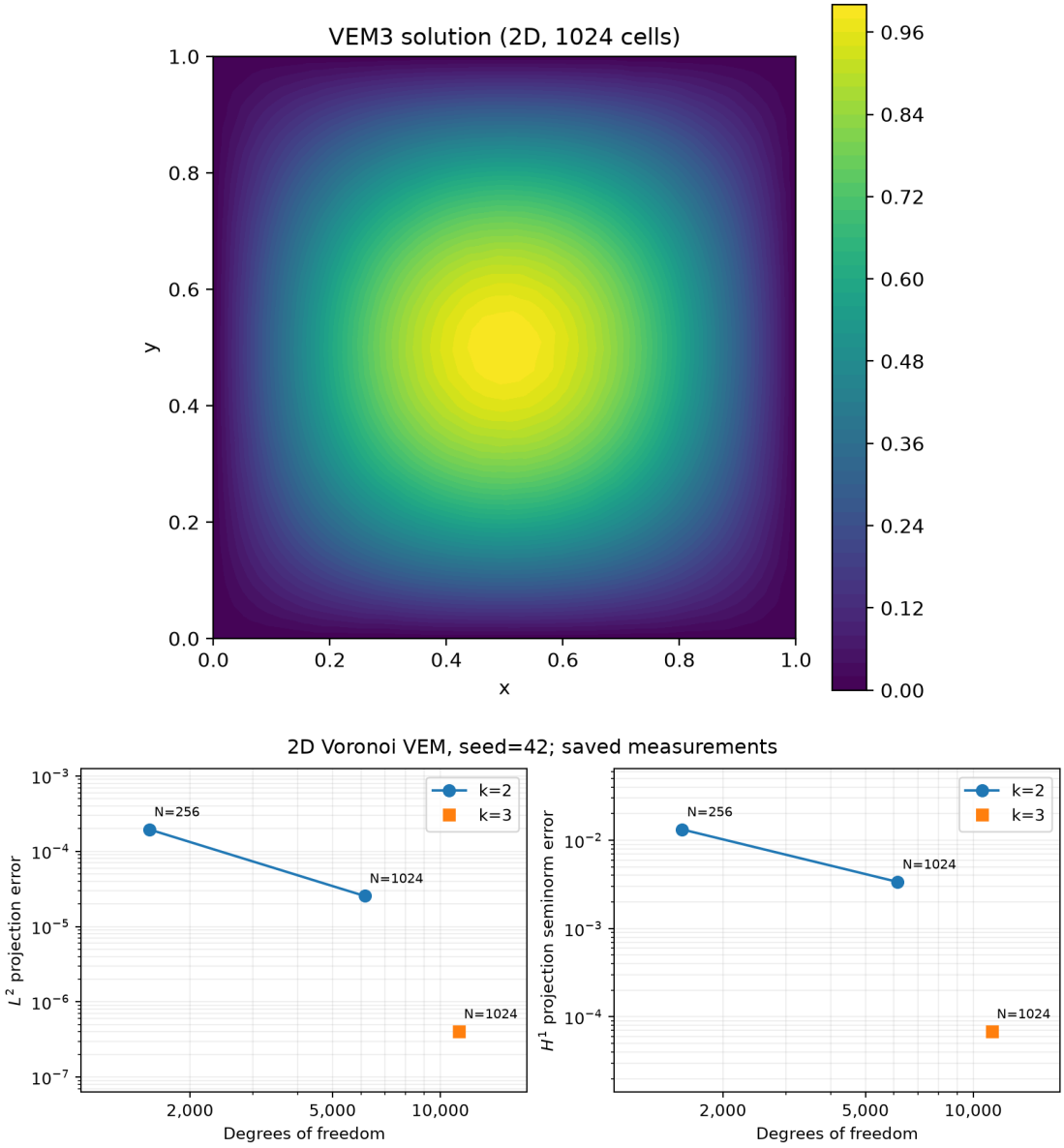


Figure 10.1: 2D cubic VEM solution (1024 cells) and error comparison from saved results.

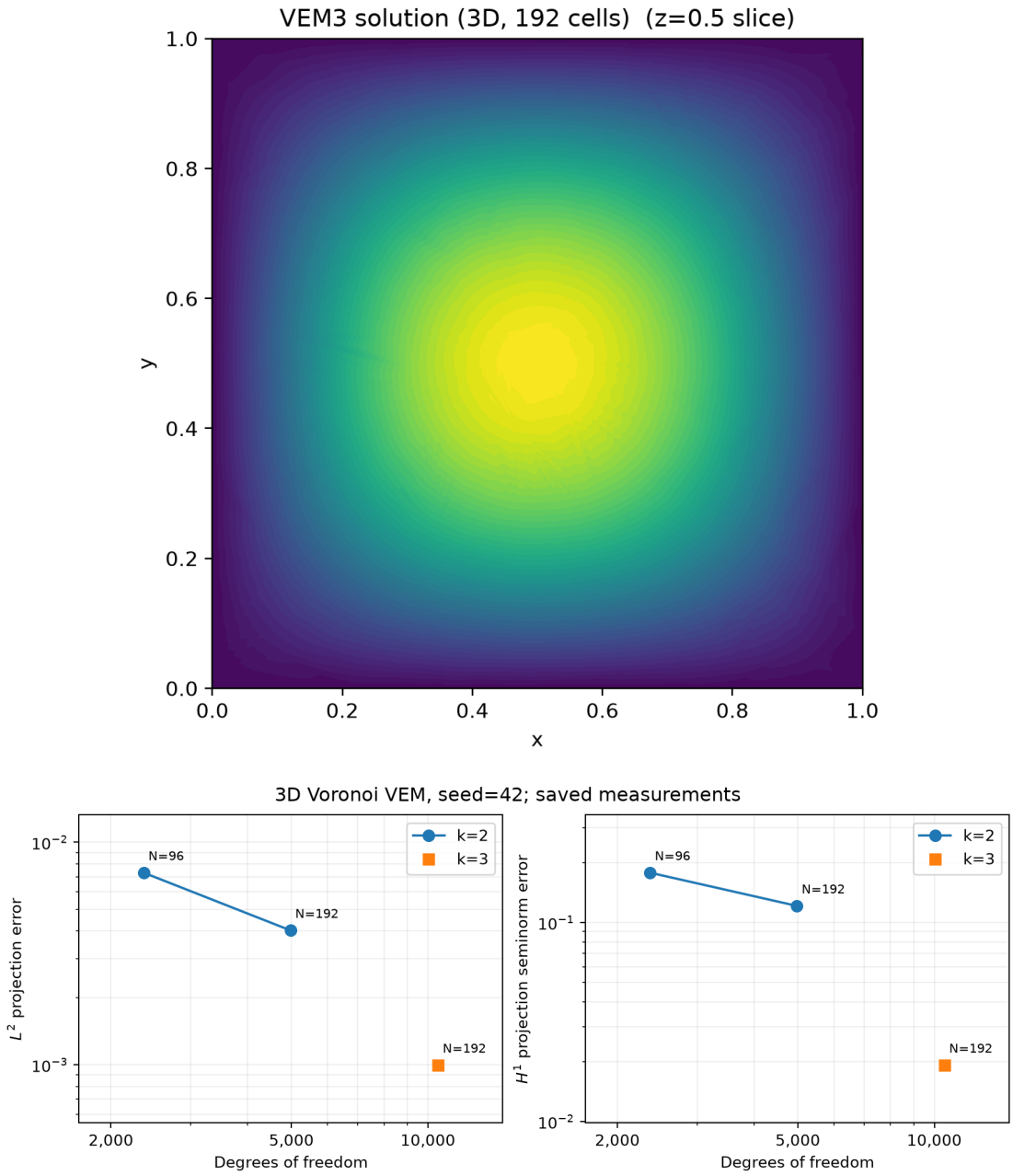


Figure 10.2: 3D cubic VEM solution (192 cells) and error comparison from saved results. The solution shows the actual $z = 0.5$ section.

From the project root, reproduce the default fine-grid examples and check the implementation:

```
python examples/vem_poisson.py
python examples/vem_poisson_3d.py
python tests/run_tests.py
```

10.4 Example 4: The Low-Frequency Helmholtz Equation

The Helmholtz equation $-\Delta u - k^2 u = f$ is a built-in problem (Chapter 6.3), so solving it costs exactly as much code as solving the Poisson equation:

```python
import numpy as np
from MosicaFE import (FESpace, HelmholtzProblem, Mesh, convergence_study,
                      estimate_convergence_rate, run, solve_helmholtz)

k = 1.0                                          # constant wavenumber
exact = lambda x: np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
grad = lambda x: np.array([np.pi*np.cos(np.pi*x[0])*np.sin(np.pi*x[1]),
                           np.pi*np.sin(np.pi*x[0])*np.cos(np.pi*x[1])])
source = lambda x: (2*np.pi**2 - k**2) * exact(x)   # manufactured solution

mesh = Mesh.rectangle(nx=32, ny=32)

# (a) one line
u = solve_helmholtz(mesh, element="P2", f=source, k=k)

# (b) step by step: mesh / element / solver / visualization
V = FESpace.lagrange(mesh, degree=2)             # or FESpace.vem(mesh)
problem = HelmholtzProblem(V, f=source, g=lambda x: 0.0, k=k)
u = problem.solve(method="direct")
print(problem.compute_errors(u, exact, grad))

# (c) by registry name
u = run("helmholtz", mesh, element="VEM", f=source, k=k)
```

Listing 10.5: Constant wavenumber, three equivalent spellings

Output on the 32×32 mesh (P2, $kh = 0.044$, $k^2/\lambda_{1,h} = 0.051$):

```
{'L2': 6.88e-05, 'Linf': 2.41e-03, 'H1': 7.38e-03}
```

The error norms are the ones of the Poisson problem on the same mesh, because in this frequency range the two discrete operators differ by a bounded perturbation of the mass matrix. What changes is the resolution requirement, and the example below makes it visible: the same code, the same tolerance, but the mesh is refined until kh is small enough for the asymptotic order to show up.

```python
def mesh_factory(level):
    return Mesh.rectangle(nx=4 * 2**level, ny=4 * 2**level)

def problem_factory(mesh):
    return HelmholtzProblem(FESpace.lagrange(mesh, 2), f=source,
                            g=lambda x: 0.0, k=k)

h, errors = convergence_study(mesh_factory, problem_factory, exact, grad,
                              refinement_levels=4, solver="direct")
```

```
print("L2 rate:", estimate_convergence_rate(h, errors["L2"], use_last=3))
```

Listing 10.6: Convergence of the Helmholtz solver

```
  level 0: h=0.35355  L2=3.5219e-02  H1=4.4393e-01  Linf=1.4645e-01
  level 1: h=0.17678  L2=4.4391e-03  H1=1.1632e-01  Linf=3.8060e-02
  level 2: h=0.08839  L2=5.5175e-04  H1=2.9416e-02  Linf=9.6074e-03
  level 3: h=0.04419  L2=6.8753e-05  H1=7.3752e-03  Linf=2.4076e-03

L2 rate: 3.006
```

Two numbers should be watched in such a table: the observed order (3.006, i.e. the P2 order of (3.20)) and the resolution kh, which here runs from 0.25 down to 0.031. What happens when the mesh is too coarse for the wave is discussed in Chapter 6; the verification data for $k = 1$ on four element families are collected in Table 6.1.

10.5 Example 5: The Reaction-Diffusion Equation

```
import numpy as np
from MosicaFE import FESpace, Mesh, ReactionDiffusionProblem

c = 10.0
exact = lambda x: np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
source = lambda x: (2*np.pi**2 + c) * exact(x)

mesh = Mesh.rectangle(nx=32, ny=32)
V = FESpace.lagrange(mesh, degree=2)
problem = ReactionDiffusionProblem(V, f=source, g=lambda x: 0.0,
                                   reaction_coeff=c)
u = problem.solve()
print(problem.compute_error(u, exact, "L2"))     # ~6.8e-05
```

Listing 10.7: The built-in reaction-diffusion problem

Chapter 11

Advanced Topics

11.1 Accuracy Pitfalls and Lessons Learned

While the three programs were being merged into a single library, four classes of problems surfaced in practice. None of them was a "small numerical accuracy issue": each of them produced wrong results or destroyed the convergence order. They are recorded here both to explain why MosicaFE is written the way it is and to help readers inspect their own codes.

11.1.1 Pitfall 1: Dirichlet Elimination Without the Right-Hand-Side Correction

Symptom: everything is fine for homogeneous boundary conditions, but the solution is wrong for inhomogeneous ones.

Diagnosis: run the linear patch test (equation (6.2)). When the exact solution belongs to the discrete space, the numerical solution must be exact.

Conclusion: see equation (6.1); step (1) cannot be omitted. MosicaFE turns this into a test in `tests/test_core_spaces.py`.

11.1.2 Pitfall 2: Insufficient Quadrature Order for Tensor-Product Elements

Symptom: the computed L^2 convergence order for 3D hexahedral Q1 elements is -0.11.

Diagnosis: the integrand of the Q1 stiffness matrix is a quadratic polynomial in each direction, and with only one Gauss point per direction the rule is exact only up to degree 1; the under-integration makes the cell matrices non-positive-definite and systematically underestimates the energy.

Conclusion: use `max(2,2*p)` as the quadrature order for tensor-product elements (at least p points per direction); see the caution after Table 4.5.

11.1.3 Pitfall 3: Fixing the Constant in High-Order Projections

The constant row of the gradient Gram matrix is zero. Replace it using equation (3.11): vertex mean at $k = 1$, true cell mean at $k \geq 2$. Check $P_\nabla D = I$ and $P_0 D = I$ on all of $\mathcal{P}_k$, rather than linear functions alone.

11.1.4 Pitfall 4: Treating Moments as Point Values

A moment is $|E|^{-1} \int_E u m_\alpha$, not the value at the entity center. Use the L^2 projection in equation (3.16) for load and mass matrices. The constant-function check needs `V.interpolate` with

`lambda x: 1.0`, not an array of ones. Nonzero Dirichlet data must also set the normalized boundary-face moments.

11.1.5 Pitfall 5: Repeated Indices in `numpy` Assignment

Symptom: `b[dofs] += local` disagrees with the sparse matrix assembly, and the row sums of the mass matrix do not match the load vector.

Diagnosis: when `dofs` contains repeated indices (for instance when the mesh has duplicate vertices), the buffered assignment of `numpy` keeps only the last value, whereas COO assembly sums them.

Conclusion: always use `numpy.add.at`.

11.2 Mesh Refinement Strategies

- **Uniform refinement**: `mesh.refine_uniform()`, multiplying the number of triangles by 4 and of hexahedra by 8;
- **Adaptive refinement (planned)**: based on the residual-type error indicator

$$\eta_K = h_K \|f + \Delta u_h\|_{L^2(K)} + h_K^{1/2} \|[\partial_n u_h]\|_{L^2(\partial K)} \tag{11.1}$$

  cells with $\eta_K > \theta \max_K \eta_K$ are marked for subdivision. The `Mesh` of MosicaFE supports ragged cells, so local refinement is feasible as far as the data structure is concerned; what is missing is the refinement implementation.

11.3 Performance and Vectorization

(1) assembly accumulates COO triplets and converts them to CSR once, at the end;

(2) basis-function evaluations on the reference element are cached per quadrature point (`LagrangeSpace._quadrature_data`), avoiding repeated work in every cell;

(3) for several physical problems on the same mesh, the result of `stiffness_matrix()` can be reused;

(4) for large problems, switching to an iterative solver is recommended;

(5) for further speedup the cell loop can be vectorized with `numba` or by "grouping cells by type and applying batched tensor operations".

11.4 Relation to Other Libraries

MosicaFE deliberately stays "small, readable and modifiable":

- compared with FEniCS / Firedrake, MosicaFE is not a general-purpose finite element platform but a **readable implementation for teaching and research**; the price is that only structured mesh generation and a handful of element families are supported;
- compared with `fealpy`, MosicaFE focuses more narrowly on "a side-by-side comparison of three discretization approaches: FEM, VEM and RT0";
- compared with the three original scripts, MosicaFE turns them into one set of interfaces, unifies meshes and solvers, and adds verification.

Chapter 12

Extending the Framework

12.1 Adding a New Element Type

The steps are:

(1) create `MosicaFE/spaces/yourspace.py` and derive from `BaseFESpace`;

(2) implement `_build_dof_map()`: build the map from cell-local degrees of freedom to global ones, and fill in `n_dofs`, `dof_support` and `dof_is_interior`;

(3) implement `stiffness_matrix()`, `mass_matrix()` and `load_vector(f)`;

(4) implement `interpolate(f)` and `evaluate(cell,u,points)`;

(5) register the new family in `FESpace.FAMILIES` in `spaces/factory.py`;

(6) add a patch test and a convergence-order test under `tests/`.

```python
from MosicaFE.spaces.base import BaseFESpace

class MySpace(BaseFESpace):
    family = "myspace"
    has_explicit_basis = True

    def _build_dof_map(self):
        # fill self.dof_map, self.n_dofs, self.dof_support, self.dof_is_interior
        ...

    def reference_nodes(self, cell_idx):
        # reference coordinates of local DOFs
        ...

    def eval_basis(self, xi):
        ...

    def eval_basis_grad(self, xi):
        ...

    def stiffness_matrix(self):
        return self._assemble(self._local_stiffness, symmetric=True)

    def mass_matrix(self):
        return self._assemble(self._local_mass, symmetric=True)

```

```
    def load_vector(self, f):
        return self._assemble_vector(lambda c: self._local_load(c, f))

    def interpolate(self, f):
        return self._evaluate_function(f, self.dof_coords)
```

Listing 12.1: Skeleton of an explicit-basis space

Key Point

If the basis functions of the new element are explicit (higher-degree Lagrange elements, DG elements, or other tensor-product families), `spaces/lagrange.py` is a good model: it already implements the generic assembly loop "evaluate on the reference element, then transform cell by cell", so usually only `eval_basis` and the degree-of-freedom layout have to be supplied.

12.2 Adding a New Equation

```
from MosicaFE.physics.base import BasePhysics
from MosicaFE.physics.registry import register_problem

@register_problem("my_pde")
class MyProblem(BasePhysics):
    """Weak form: a(u, v) = l(v)."""

    def __init__(self, space, f=None, g=None, **kwargs):
        super().__init__(space, f=f, g=g, **kwargs)
        # store problem specific data here

    def assemble(self):
        K = self.space.stiffness_matrix()
        M = self.space.mass_matrix()
        A = K + 3.0 * M                        # example combination
        b = self.space.load_vector(self.f)
        return self.apply_boundary_conditions(A, b)
```

Listing 12.2: Template for a new PDE

After registration, `available_problems()` contains `"my_pde"`, and both `create_problem("my_pde",V,...)` and `run("my_pde",mesh,element="VEM",...)` are immediately available.

The built-in equations are written exactly after this template, and they are the cheapest way to calibrate how much code a new equation really needs: `PoissonProblem` and `ReactionDiffusionProblem` are 8 and 14 lines long, and the Helmholtz problem of Chapter 6.3 differs from the Poisson problem by a single term, $A = \kappa K - k^2 M$.

Caution

The template above assumes that the right-hand side *is* the load vector of a function, `space.load_vector(f)`. Not every physical source fits that shape: a point source $\delta(x - x_0)$, a boundary term $\int_{\Gamma_N} hv \, \mathrm{d}s$, or a discontinuous coefficient must be assembled separately and added to `b` before `apply_boundary_conditions` is called. A complete worked example is `examples/custom_pde.py`, which registers `PointSourcePoissonProblem(`$-\Delta u = f + \delta(x - x_0)$`)`: for a nodal space the identity $\varphi_i(x_0) = \delta_{ij}$ makes the Dirac functional a single entry $b_j \mathrel{+}= 1$.

12.3 Adding a New Mixed Formulation (Using RT0×P0 as a Template)

A mixed formulation (a saddle-point problem) differs from the "elliptic operator" pattern above, but the division of labour is the same: **the space computes the cell matrix blocks, and the physics layer only assembles them**. Inside the library, RT0×P0 is implemented exactly after this template and can be copied:

(1) if a new facet degree-of-freedom layout is needed, obtain the facet topology from `core/facets.py` first (global facet indices, `cell_facets`, `signs`);

(2) implement the "block" interfaces in the space: here `flux_mass_matrix()` ((Φ, Φ)) and `divergence_matrix()` ((P, Φ)), together with `pressure_load_vector(f)`;

(3) assemble the blocks into a saddle-point matrix in the physics layer with `bmat` (from `MosicaFE._compat`): `bmat([[M,-B.T],[B,None]])`;

(4) to solve, use `"direct"`, `"schur"` (static condensation) or `Solver.minres` (symmetric indefinite);

(5) put the error analysis in `postprocess/error.py`, following `mixed_error_norms` (one set of norms for the pressure and one for the flux).

Key Point

The `physics` layer contains only 6 lines of assembly code (see `assemble` in `physics/mixed_poisson.py`), while the cell geometry and the numerical integration stay entirely in `spaces/rt0.py`. Replacing the mixed element (for instance by BDM1, or by the $Q_2 \times Q_1$ pair for Stokes) therefore only requires providing the new blocks; the physics layer and the postprocessing can be reused unchanged.

12.4 Adding a New Mesh Type

(1) add a class method to `core/mesh.py` returning `Mesh(nodes,elements,element_type, name=...)`;

(2) 2D cells are automatically reordered counterclockwise according to their signed area, and 3D tetrahedra according to their volume;

(3) if a 3D cell is neither a tetrahedron nor a hexahedron, supply the face table of every cell (a list of local vertex indices) through `faces=...`.

```python
import numpy as np
from MosicaFE import Mesh

# a two-element hexagonal domain (arbitrary polygons are fine)
nodes = np.array([[0.0, 0.0], [1.0, 0.0], [1.5, 0.9],
                  [0.5, 1.5], [-0.5, 0.9], [0.5, 0.6]])
elements = [[0, 1, 2, 5], [0, 5, 3, 4],
            [1, 2, 3, 5], [0, 3, 4, 5]]

mesh = Mesh.from_arrays(nodes, elements, boundary_nodes=[0, 1, 2, 3, 4])
print(mesh.summary())
```

Listing 12.3: Custom cell shapes

12.5 Development Workflow and Testing Requirements

The recommended workflow is:

(1) write a **failing** test first (patch test, convergence order, symmetry, ...);

(2) then implement the feature;

(3) run `tests/run_tests.py` and `tests/run_checks.py`;

(4) update the progress and verification records under `progress/`.

Every new feature should cover at least:

- cell level: Kronecker property of the basis functions, partition of unity, symmetry / positive definiteness of the cell matrices;
- space level: symmetry of the global matrices, the constant null space, and the consistency of the row sums of M with $b(f \equiv 1)$;
- problem level: the patch test (a linear exact solution must be reproduced exactly) and the convergence order;
- end to end: the one-line interfaces `solve_poisson` / `run` must work.

Chapter 13

Troubleshooting and FAQ

13.1 Common Issues

13.1.1 Import Errors

Symptom: `ModuleNotFoundError: No module named 'MosicaFE'`.

Cause: MosicaFE lives in `D:\mosica\MosicaFE`, so the **parent directory** `D:\mosica` has to be added to the module path.

```
# option 1: install in editable mode
cd D:\mosica\MosicaFE && python -m pip install -e .

# option 2: extend PYTHONPATH
$env:PYTHONPATH = "D:\mosica"
```

13.1.2 Wrong Convergence Order

Check the following, in order:

(1) run the patch test first (equation (6.2) in Chapter 6). If it fails, the problem is certainly in the assembly or in the boundary handling;

(2) check whether the quadrature order is sufficient (tensor-product elements are especially prone to under-integration);

(3) check whether the definition of h is consistent with the mesh refinement;

(4) check whether the exact solution is smooth enough (manufactured solutions should preferably use trigonometric functions or polynomials);

(5) for VEM, check that the stabilization option is `"trace"`.

13.1.3 Iterative Solver Does Not Converge

- verify first with a direct method that the system itself is solvable;
- CG requires a symmetric positive definite matrix: saddle-point problems must use MINRES or a direct method;
- increase `maxiter` and relax `tol`;
- if the condition number is very large, consider scaling or preconditioning.

13.1.4 Out of Memory

- make sure sparse matrices are being used (`scipy` installed);
- three-dimensional problems grow quickly: n^3 nodes correspond to roughly $6n^3$ tetrahedra;
- switching to an iterative method avoids the fill-in of direct factorizations.

13.2 Frequently Asked Questions

Q: How do I choose the polynomial degree? A: In the vast majority of cases P2 / Q2 offers the best accuracy-to-cost ratio; start with P1 for a quick check, and switch to quadratic elements once the solution is smooth. VEM accepts any positive integer k through `degree=k`; high-order monomial matrices require attention to conditioning.

Q: When should I use an iterative solver? A: Below roughly 2×10^4 degrees of freedom a direct solver is the least trouble; for larger problems switch to CG (symmetric positive definite) or GMRES (general).

Q: What exactly is the difference between VEM and FEM? A: VEM uses implicit basis functions through polynomial projections and stabilization, allowing admissible polygons, polyhedra and varying face counts. The triangular first-order case is a special case related to P1. Higher-order VEM includes moments and is not identical to the same-degree Lagrange element. See Section 4.1.2 for the geometry requirements.

Q: Why does the RT0 solution return two arrays? A: The mixed formulation solves for the flux q_h and the pressure p_h simultaneously. `Q[c,i]` is the outward normal flux on the i-th facet of cell c (numbered so that facet i is the one "not containing vertex i"), and `P[c]` is the constant pressure on that cell.

Q: Does RT0 require installing or linking an external program? A: No. RT0×P0 is a native method of MosicaFE: the explicit basis functions $\varphi_i = (x - a_i)/(d|K|)$ and the exact mass matrix are in `spaces/rt0.py`, the facet topology and orientation are in `core/facets.py`, the saddle-point assembly is in `physics/mixed_poisson.py`, and the error norms are in `postprocess/error.py`. The dependencies of the library are still just numpy (scipy / matplotlib optional), and after `import MosicaFE` the call `FESpace.rt0(mesh)` works directly.

Q: Can time-dependent problems be solved? A: The current version handles steady elliptic problems only. For time-dependent problems the method of lines is recommended: space discretization gives $M\dot{u} + Au = b$, and the time direction is advanced with an explicit or implicit scheme of your choice. `mass_matrix()` is already provided.

Q: Can nonlinear problems be solved? A: The framework targets linear problems. For nonlinear problems, write a Newton iteration in the outer loop and use MosicaFE to assemble and solve the Jacobian system at every step.

Q: How do I export results to ParaView? A: Use `space.nodal_values(u)` to obtain nodal values, combine them with `mesh.nodes` and `mesh.elements`, and write a `.vtu` or `.vtk` file yourself; alternatively add an export function to `postprocess`.

Appendix A

API Reference

A.1 Top-Level Module `MosicaFE.`

Name	Description
`Mesh`	mesh object and factory methods
`FESpace`	space factory: `lagrange` / `vem` / `rt0` / `create`
`FacetTopology`, `build_facet_topology`	facet topology and orientation (for $H(\mathrm{div})$ spaces)
`QuadratureRule`	quadrature rules on reference elements
`cell_quadrature(...)` / `simplex_quadrature(...)`	quadrature points and weights on physical cells / arbitrary simplices
`BaseFESpace`	base class of the spaces
`LagrangeSpace` / `VEMSpace` / `RT0Space`	the three space families
`rt0_flux_basis`, `rt0_mass_matrices`	explicit RT0 basis functions and exact mass matrices
`BasePhysics`	base class of the physics problems
`PoissonProblem`	$-\nabla \cdot (\kappa \nabla u) = f$
`MixedPoissonProblem`	RT0×P0 mixed formulation
`ReactionDiffusionProblem`	$-\Delta u + cu = f$
`HelmholtzProblem`	$-\nabla \cdot (\kappa \nabla u) - k^2 u = f$ (constant k)
`register_problem`, `create_problem`, `available_problems`	PDE registry
`Solver`	linear solvers (direct / cg / gmres / bicgstab / minres / auto)
`compute_error` / `compute_errors`	error norms
`mixed_error_norms`	pressure / flux error norms for RT0×P0
`convergence_study`	convergence studies
`estimate_convergence_rate`	convergence-order estimation
`plot_solution` / `plot_mesh` / `plot_error`	visualization
`plot_convergence` / `plot_pressure_2d` / `plot_flux_2d`	visualization
`solve_poisson`	one-line Poisson solver
`solve_helmholtz`	one-line Helmholtz solver
`run`	solve any registered PDE by name

A.2 `Mesh`

Method / attribute	Description
`Mesh.interval(a,b,nx)`	one-dimensional interval mesh
`Mesh.rectangle(...)`	two-dimensional rectangular mesh (`triangle`/`quad`)
`Mesh.box(...)`	three-dimensional box mesh (`tet`/`hex`)
`Mesh.pentagon(nx,ny)`	pentagon mesh
`Mesh.voronoi_polygon(...)`	Voronoi polygonal mesh
`Mesh.voronoi_polyhedron(...)`	Voronoi polyhedral mesh
`Mesh.from_arrays(...)`	custom mesh
`mesh.refine_uniform()`	uniform refinement
`mesh.get_mesh_size()`	maximum cell diameter h
`mesh.get_cell_vertices(i)`	coordinates of the cell vertices
`mesh.get_cell_measure(i)`	cell area / volume
`mesh.get_cell_diameter(i)`	cell diameter
`mesh.get_cell_centroid(i)`	cell centroid
`mesh.boundary_nodes`	boundary vertex indices
`mesh.boundary_facets` / `mesh.facets`	facet topology
`mesh.facet_topology`	`FacetTopology`: global facet indices, shared facets, outward normal signs
`mesh.summary()`	one-line summary

A.3 FESpace and the Space Methods

Method	Description
`FESpace.lagrange(mesh, degree)`	continuous Lagrange (P1/P2/Q1/Q2)
`FESpace.vem(mesh,degree=k)`	enhanced VEM (positive integer k)
`FESpace.rt0(mesh)`	RT0×P0 mixed space
`FESpace.create(mesh,family, degree)`	create by name
`FESpace.available()`	available space families
`V.n_dofs`	number of degrees of freedom
`V.get_cell_dofs(i)`	cell-local → global degrees of freedom
`V.get_dof_coordinates(k)`	coordinates of a degree of freedom
`V.boundary_dofs()`	Dirichlet degrees of freedom
`V.eval_basis(xi)` / `V.eval_basis_grad(xi)`	basis functions on the reference element
`V.stiffness_matrix()`	stiffness matrix
`V.mass_matrix()`	mass matrix
`V.load_vector(f)`	load vector
`V.interpolate(f)`	interpolation
`V.evaluate(c,u,pts)`, `V.evaluate_gradient(...)`	reconstruction and evaluation inside a cell
`V.nodal_values(u)`	nodal values (for visualization)
`V.flux_mass_matrix()`	RT0: the (Φ,Φ) block (exact closed-form mass matrix)
`V.divergence_matrix()`	RT0: the (P,Φ) block (divergence, always ±1)
`V.pressure_load_vector(f)`	RT0: right-hand side $\int_K f$ of the pressure equation

Method	Description
`V.interpolate_pressure(f)` / `V.interpolate_flux(q)`	RT0: interpolation of each of the two fields
`V.evaluate_flux(c,Q,pts)`	RT0: reconstruction of q_h inside a cell
`V.locate_cell(x)`	RT0: locate the cell containing a point
`V.n_flux` / `V.n_pressure` / `V.topology`	RT0: sizes and facet topology

A.4 Physics Problems

Method	Description
`PoissonProblem(V,f,g,kappa)`	constructor
`problem.assemble()`	returns (A, b) (boundary conditions included)
`problem.solve(method)`	assemble and solve
`problem.compute_error(u, exact,norm)`	a single error norm
`problem.compute_errors(u, exact,grad)`	all error norms
`HelmholtzProblem(V,f,g,k, kappa)`	constructor, k and κ are real constants
`helmholtz.k`, `helmholtz.kappa`	the stored coefficients ($A = \kappa K - k^2 M$)
`solve_helmholtz(mesh, element,f,g,k)`	one-line Helmholtz solve (rejects `"RT0"`)
`MixedPoissonProblem(...)`	returns (Q, P)
`mixed.error_norms(P,Q,p_ exact,q_exact)`	pressure / flux errors
`mixed.check_conservation()`	cell-wise mass conservation error
`mixed.flux_at_ points(points)`	flux reconstruction at given points (cell location is automatic)
`mixed.cell_flux(cell)`	flux vector at the cell centroid (for arrow plots)
`mixed.solve(method)`	three ways to solve the saddle-point system: `direct` / `schur` / `minres`

A.5 Solvers and Postprocessing

Method	Description
`Solver.direct(A,b)`	direct method
`Solver.conjugate_ gradient(...)`	CG
`Solver.gmres(A,b,tol)`	GMRES
`Solver.bicgstab(A,b,tol)`	BiCGSTAB
`Solver.minres(A,b,tol)`	MINRES
`Solver.solve(A,b,method)`	unified entry point
`Solver.apply_dirichlet(...)`	impose Dirichlet data
`Solver.recommend(n, symmetric)`	recommend a solver

Method	Description
`convergence_study(...)`	convergence study
`estimate_convergence_rate(h,e)`	convergence order
`plot_solution(u,mesh,V,...)`	plot the solution
`plot_mesh(mesh,...)`	plot the mesh
`plot_error(u,mesh,space,exact)`	plot the error
`plot_convergence(h,errors)`	plot convergence curves
`plot_pressure_2d(mesh,P)`	plot the RT0 pressure
`plot_flux_2d(space=V,Q=Q)`	plot the flux (reconstructed from q_h at cell centroids)
`compare_solutions(entries)`	side-by-side comparison

Appendix B

Mathematical Notation

B.1 General Notation

Symbol	Meaning
Ω	computational domain
$\partial\Omega$	boundary of the domain
d	spatial dimension (1, 2, 3)
h	mesh size (maximum cell diameter)
K	mesh cell
$\hat{K}$	reference cell
κ	diffusion coefficient (constants $\kappa > 0$ in the built-in problems)
k	wavenumber of the Helmholtz problem (constant; k^2 is the shift against λ_1)
λ_1, $\lambda_{1,h}$	first Dirichlet eigenvalue of $-\Delta$, exact and discrete
λ	generic eigenvalue (generalized problem $Kv = \lambda Mv$)
n	number of degrees of freedom
$\mathcal{D}$	set of Dirichlet degrees of freedom

B.2 Function Spaces

Symbol	Meaning
V	infinite-dimensional trial / test space
$V_h \subset V$	finite-dimensional discrete space (finite or virtual elements)
$H^1(\Omega)$	first-order Sobolev space
$H(\mathrm{div};\Omega)$	space of vector fields with square-integrable divergence
$L^2(\Omega)$	space of square-integrable functions
$\mathcal{P}_k$	polynomials of degree at most k
Q_k	tensor-product polynomials of degree at most k in each variable

B.3 Operators and Norms

Symbol	Meaning
∇u	gradient
$\nabla \cdot q$	divergence
Δu	Laplace operator
φ_i / $\hat{\varphi}_i$	basis functions on the physical / reference cell
Π^{∇}	gradient projection operator of VEM
Π_{dof}	matrix representation of the projection on the degrees of freedom
A_{proj}, S_{stab}	consistency and stabilization parts of the VEM cell matrix
J_K	Jacobian matrix of the reference-to-physical mapping
$\|e\|_{L^2}$, $\|e\|_{H^1}$, $\|e\|_{L^\infty}$	error norms
q, p	flux and pressure in the RT0 mixed formulation
A, b	matrix and right-hand side of the discrete linear system

Appendix C

Bibliography

Bibliography

[1] S. C. Brenner and L. R. Scott, *The Mathematical Theory of Finite Element Methods*, 3rd ed., Springer, 2008.

[2] A. Ern and J.-L. Guermond, *Theory and Practice of Finite Elements*, Springer, 2004.

[3] A. Logg, K.-A. Mardal, and G. Wells (eds.), *Automated Solution of Differential Equations by the Finite Element Method*, Springer, 2012.

[4] L. Beirão da Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. D. Marini, and A. Russo, "Basic principles of virtual element methods", *Mathematical Models and Methods in Applied Sciences*, 23(1):199–214, 2013.

[5] L. Beirão da Veiga, F. Brezzi, L. D. Marini, and A. Russo, "The Hitchhiker's Guide to the Virtual Element Method", *Mathematical Models and Methods in Applied Sciences*, 24(8):1541–1573, 2014.

[6] L. Beirão da Veiga, F. Brezzi, L. D. Marini, and A. Russo, "Virtual element methods for general second order elliptic problems on polygonal meshes", *SIAM Journal on Numerical Analysis*, 2013.

[7] F. Brezzi and M. Fortin, *Mixed and Hybrid Finite Element Methods*, Springer, 1991.

[8] P.-A. Raviart and J.-M. Thomas, "A mixed finite element method for 2nd order elliptic problems", in *Mathematical Aspects of Finite Element Methods*, Springer, 1977.

[9] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.